

Evaluation of Deep Learning-Based SLAM and Traditional SLAM Method

By

Bagadi, Tarun Kumar

MSc Aerial Robotics Dissertation



Department of Engineering Mathematics

UNIVERSITY OF BRISTOL

&

A MSc dissertation submitted to the University of Bristol
in accordance with the requirements of the degree of
MASTER OF SCIENCE IN AERIAL ROBOTICS in the
Faculty of Engineering.

December 25, 2023

Declaration of own work

I declare that the work in this MSc dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Research Degree Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, the work is the candidate's own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in the dissertation are those of the author.

Tarun Kumar Bagadi

December 25, 2023

Acknowledgement

Without the guidance, and help of the following people, this work would not have been possible.

I express profound gratitude towards my supervisor, Dr. Walterio Mayol-Cuevas, for his consistent support during the course of my dissertation. The constant encouragement and willingness to offer detailed comments on my work proved to be of immense value. I express my gratitude for the amount of time and effort he dedicated to assist me in overcoming the obstacles encountered during my research endeavor and attaining my objectives.

I would also like to extend my appreciation to Dr. Tom Richardson for his tremendous assistance and unwavering support, which constantly boosted my progress.

In conclusion, I would like to express my deepest appreciation to my parents for their persistent and ever ending motivation and assistance, which have played a pivotal role in my achievements. I am indebted to their contributions since they played an indispensable role in enabling me to achieve this significant accomplishment.

Abstract

SLAM technology is extensively utilized across several domains, spanning self-driving vehicles, mobile robots, unmanned aerial vehicles, underwater robotics, and interplanetary rovers. One of the primary difficulties encountered in the field of simultaneous localization and mapping (SLAM) is the precise determination of the robot's position. The present dissertation aims to explore and assess a resilient real-time VO technique known as ORB-SLAM2[1].

Traditional SLAM techniques mainly depend on geometric restrictions that are obtained from images. On the contrary, approaches for visual odometry that are based on deep learning have the capability to leverage the abundant information included in the entire image and successfully capture complex motion dynamics.[2]. The secondary aim of this research is to explore a visual odometry technique called Deepvo,[2] which is based on deep learning principles and compare the performance of the above mentioned two techniques. The experimental results will be analyzed using established evaluation metrics, such as absolute trajectory (ATE), and relative pose error(RPE) .

The presence of dynamic objects in the environment surrounding robots has the potential to impact their visual capabilities. The utilization of semantic segmentation and incorporation of semantic information might improve robot localization by mitigating uncertainty. The third aim is to study and examine the use of the Deeplabv3+ [3] deep neural network model for the purpose of semantic segmentation of images, to efficiently detect dynamic objects, and improving the performance of VSLAM in highly dynamic scenarios and will investigate the effectiveness of combining semantics with ORB-SLAM2 to compute odometry.

The results obtained from the evaluation performed on the KITTI dataset[4] suggest that the traditional approach, ORB-SLAM2, outperforms the deep learning methodology Deep VO in the computation of visual odometry. The study also revealed that the removal of dynamic objects from images may increase the performance of VSLAM pipelines in dynamic situations. However, additional validation and testing is required in order to support this hypothesis.

Number of words in the dissertation: 14059 words.

Contents

	Page
1 Introduction	8
1.1 Aims and Objectives	9
1.2 Framework of VSLAM	10
1.3 Visual Odometry	11
1.4 Background	13
2 Literature Review	17
2.1 ORB SLAM2	17
2.2 Deep VO	24
2.3 Semantic Segmentation	30
3 Research Methodology	32
3.1 Experimental setup	32
3.2 Dataset	36
3.3 Error Metrics	37
4 Results	38
4.1 Comparision of Orbslam and Deepvo	38
4.2 Semantic Segmentation and Masked ORBSLAM2	43
4.3 Comparison with simple VO	50
5 Discussion and Conclusion	54
A Appendix	57
References	74

List of Abbreviations

SLAM Simultaneous Localization and Mapping

VSLAM Visual Simultaneous Localization and Mapping)

ORB-SLAM Oriented FAST and Rotated BRIEF Simultaneous Localization and Mapping.

VO Visual Odometry

DL-VO Deep Learning Visual Odometry

ANN Artificial Neural Network

CNN Deep Learning Convolutional Neural Network

RNN Deep Learning Recurrent Neural Network

LSTMs Long Short-Term Memory networks

RCNNs Deep Recurrent Neural Networks

BA Bundle Adjustment

DNNs Deep Neural Networks

GPS Global Positioning System

SIFT Scale-Invariant Feature Transform

SURF Speeded-Up Robust Features

FAST Features from Accelerated Segment Test

ORB Oriented FAST and Rotated BRIEF

ATE Absolute Trajectory Error

RPE Relative Pose Error

List of Tables

4.1 ORBSLAM and DeepVO Error Comparision in Meters. 43

List of Figures

1.1	Frame Work of VSLAM [14]	10
1.2	Principle of an artificial neuron [44]	15
2.1	ORB SLAM2 System Threads and Modules [1]	18
2.2	Architecture of the proposed RCNN-based monocular VO system [2]	25
2.3	Configuring of the CNN [2]	26
2.4	Folded and unfolded LSTMs and internal structure of its unit. [2]	29
2.5	Deeplabv3+ network [3]	31
4.1	Trajectory and Error plots for KITTI [4] sequence 4	39
4.2	Trajectory and Error plots for KITTI [4] sequence 5	40
4.3	Trajectory and Error plots for KITTI [4] sequence 6	41
4.4	Trajectory and Error plots for KITTI [4] sequence 7	42
4.5	Statistical data on Kitti [4]Sequences	44
4.6	Original and Semantic output from Deeplabv3+[3]	45
4.7	Original and masked image	46
4.8	Comparison of ORB Features on masked and orginal	47
4.9	Comparison of masked ORBSLAM on modified sequence 01	48
4.10	Comparison of masked ORBSLAM on modified sequence 02	48
4.11	ORBSLAM2 and Masked ORBSLAM2 on modified sequence 03	49
4.12	Error comparison of ORB SLAM2 and Masked ORB SLAM2 on Apollospace dataset	49
4.13	Comparision of simple VO with ORBSLAM2 and DEEPVO on KITTI dataset [4] .	50
4.14	Images for modified Sequence 02	51
4.15	Few Images for modified Sequence 01	52
4.16	Images for apollo space dataset	53

1 Introduction

Autonomous robots are intelligent machines capable of autonomous navigation and operation, independent of any human intervention. In order to achieve this, robots should have awareness of their surroundings and accurately determine their position within the surroundings. SLAM is a methodology performed by robots to construct a spatial map of their environment while simultaneously determining their own position within the map in real time. The solution was largely formulated by Hugh Durrant Whyte and John J. Leonard in 1991 [5], building upon the research conducted by Smith et al. in 1990 [6].

SLAM has been the field of research over long period of time, resulting in the development of numerous strategies aimed at enhancing its performance. The majority of these methodologies requires costly sensors, such as LiDARs, making the process costly and unfeasible for a wide range of applications. VSLAM has demonstrated superior cost-effectiveness and reliability because it can be implemented with the use of cameras. VSLAM has emerged as an optimal choice for robots. VSLAM proves to be highly advantageous in the context of robotic exploration within enclosed or subterranean settings, such as interior spaces, caves, mines, nuclear power plants, and sewers. The ability of these robots to maneuver in confined areas often makes the GPS systems useless. VSLAM enables robotic systems to construct a map of their surroundings and accurately determine its position even in scenarios where GPS signals are unavailable.

VSLAM is the technique that utilizes point characteristics [7], [8] extracted from images to estimate the position and create a map. Visual Simultaneous Localization and Mapping (VSLAM) is a robust and cost-effective solution that empowers autonomous robotic systems to traverse complex surroundings without any human involvement. The field is experiencing significant development, with continuous advancements with the development of new approaches.

1.1 Aims and Objectives

1. Literature Review and Implementation

Conduct a literature review of traditional visual odometry and deep learning-based odometry techniques for SLAM and additionally implement and test the both odometry approaches.

- (a) Research and implement traditional visual odometry, ORB-SLAM2, and test it [4]dataset.
- (b) Research and implement deep learning-based visual odometry, DeepVO, and test it on Kitti [4] dataset.

2. Evaluation and Comparison

To evaluate deep learning-based odometry and traditional odometry method using the dataset

- (a) Perform a trajectory based evaluation on deep learning-based odometry and traditional Odometry method on KITTI [4] Dataset.
- (b) To identify and define error metrics for evaluating the performance of odometry methods, such as absolute trajectory error (ATE), relative pose error (RPE).
- (c) To analyze and compare the error metrics obtained from the evaluation of DeepVO and ORB-SLAM2 on the KITTI [4]dataset.

3. Semantic Segmentation

Add semantic information to visual Odometry using semantic segmentation model to track the dynamic object for visual odometry method and evaluate its impact on trajectory estimation in dynamic environment.

- (a) Implement the Deeplab v3+ [3] model for semantic segmentation.
- (b) Develop a method for masking dynamic objects and process it to the visual odometry .
- (c) Test the impact of masked images on ORBSLAM2[1] for trajectory estimation .

1.2 Framework of VSLAM

VSLAM frameworks generally have four distinct modules:- frontend visual odometry, backend optimization, loop closure detection, and mapping.[9]

The frontend visual odometry module is responsible for the estimation of the robot's position from one frame to another, as well as the determination of the position of map points. This process involves the computation of camera motion between consecutive images and the construction of local maps. The backend optimization module is responsible for refining the position information received from the frontend module. It achieves this by employing many techniques, including sliding window optimization,[10] bundle adjustment,[11] and pose graph optimization. This aids in the incorporation of errors and enhances the level of accuracy of pose estimation by rectifying cumulative errors.[12]–[14]

The loop closure detection module performs a comparison between the current data obtained from the front end and the previously recorded data in order to determine whether the robot has made it back to its original location. If a loop is identified, it rectifies the accumulated error in pose estimation. The mapping module utilizes camera position and frames to reconstruct the map. The representation of the environment is constructed by utilizing the estimated poses and detected features. [15]

The collaboration between the frontend and backend modules facilitates the estimation of the robot's posture and mapping of the surrounding environment. In parallel, the loop closure detection and mapping modules contribute to enhancing the precision and coherence of the estimated poses and maps.[14]

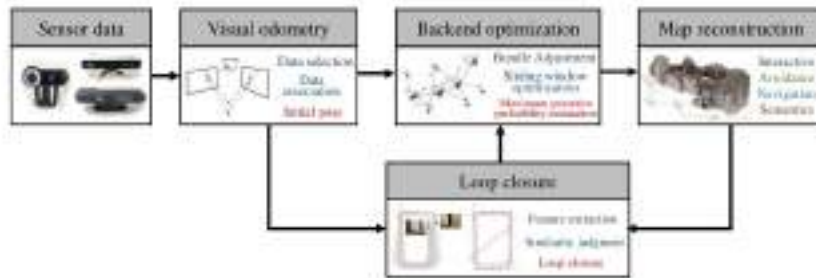


Figure 1.1: Frame Work of VSLAM [14]

1.3 Visual Odometry

Visual odometry (VO) refers to a technique used to estimate the movement of an agent, such as a robot, mobile device, or autonomous vehicle, by using images. The introduction of this concept was documented by Nister et al. in 2004.[16] The word "odometry" is similar to wheel odometry, when a vehicle determines its motion by integrating the count of wheel rotations over a certain duration. [17] Odometry can be computed using a wide variety of sensors, the most common of which are listed in [18] along with their principles. VO has several benefits in comparison to traditional approaches, including GPS, wheel odometry, and sonar localization devices.[19]. In the context of VO, the estimation of pose is achieved by the analysis of the shift in the visual content recorded by a camera in motion.

There are two primary categories of visual odometry algorithms, namely geometry-based approaches and learning-based approaches.[2], [19], [20]

1.3.1 Geometry-based Approaches

Geometric approaches serve as the fundamental basis for VO algorithms, which aim to estimate motion by extracting geometric constraints from images. Geometric-based VO algorithms have gained significant popularity due to their elegant nature, well-established foundations, and intensive research efforts.[2]

Geometric approaches are often classified into two primary types, namely Direct methods and Feature-based methods. [21]

Feature-based approaches

Feature-based approaches are often used in order to identify unique structures within images, which can then be compared and matched with corresponding structures in subsequent images. Prominent characteristics may be readily identified based on the intensity values within a small window, and they exhibit invariance in viewing angle, scale, and lighting conditions.

The estimation of the camera's pose is done by matching feature points across images. Consequently, it is essential to provide a distinct description for each feature point in order to facilitate its

recognition in future images. The SIFT[22], [23] is a frequently used feature descriptor, whereas the Fast [24] feature detector is a routinely utilized feature-point detector.

Direct approaches

In order to estimate odometry, rather than using feature extraction and matching algorithms based on features, this approach calculates the variations in appearance by analyzing the changes in pixel intensity between two images[25], which is equivalent to the functioning of optical flow algorithms.[26] These methods are often regarded as being more precise in comparison to feature-based approaches, particularly in scenarios where there is a lack of texture.[2]

Semi-direct methods aim to enhance the robustness and effectiveness of outcomes over a broader range of circumstances by combining both the feature-based approach and the direct approach.

1.3.2 Learning-based Approaches

Learning-based VO methods use deep learning techniques to extract significant characteristics from images. These features are then used to estimate the position of the camera.[27] Neural networks (NNs) [28], [29] provide a notable capability to analyze unprocessed input data, hence removing the need for feature extraction during the design phase. Learning-based VO may be categorized into two main approaches, namely supervised and unsupervised, depending on the presence or absence of ground truth data. [30], [31]

Supervised techniques depend on the use of external ground truth data to create training labels, which are then used to assess the accuracy of the model in comparison to the ground truth. The content of the training labels is subject to variation based on the specific methodology used and the particular external ground truth utilized. In the context of Deep Learning-based Visual Odometry (DL-VO), the external ground truth often comprises absolute or the relative pose between consecutive frames, hence facilitating the estimate of visual odometry.[31]

Unsupervised learning is used in scenarios when the ground truth, or the accurate annotations for the data, are unavailable. The process of generating extensive datasets for supervised learning may be both costly and time-intensive since it often requires manual annotation for each individual data instance. Unsupervised learning algorithms provide the capability to recognize patterns and

establish correlations within unlabeled data. This enables the extraction of valuable insights from the data, even in the absence of established ground truth.[30], [32]

1.4 Background

1.4.1 Feature

Scale-Invariant Feature Transform (SIFT)[23]

SIFT [23] is used in the field of computer vision for the purpose of detecting and describing local features from images. SIFT features are invariant to dimensions, orientation, and lighting, making them highly suitable for a diverse range of computer vision applications, including object identification, and 3D reconstruction.[33]

The SIFT operates by first identifying key points within the image. Key points refer to regions of local contrast that are very likely to serve as unique characteristics. The SIFT proceeds by assigning an orientation to every key point, which is determined by analyzing the local picture gradient. In conclusion, the SIFT algorithm produces a feature descriptor consisting of 128 elements for every detected key point. The characteristic is determined by calculating the local image gradient inside a certain area around the key point. This gradient is then multiplied by a Gaussian function that is centered on the key point. The normalization process is used to standardize the descriptor, while thresholds are utilized to mitigate the impact of image saturation.

The SIFT descriptor is a very used feature descriptor in the field of computer vision. The feature descriptor is well recognized for its ability to effectively handle variations in size, orientation, and lighting, while also exhibiting a notable level of individuality. Nevertheless, the SIFT is hindered by its high computing cost, rendering it impractical for real-time .[34], [35]

Speeded-Up Robust Features (SURF)[36]

The SURF algorithm is a local feature detector and descriptor that is very efficient and resilient, drawing inspiration from the SIFT method. The detection of features is accomplished by using a Fast Hessian detector at various scales. Subsequently, a descriptor consisting of either 64 or 128 elements is estimated by considering the local picture gradients. The SURF uses integral images as

a means to estimate picture convolutions, resulting in a significant enhancement in its computational efficiency. The SURF descriptors exhibit comparable invariance properties to SIFT, namely with regard to size, orientation, and illumination. [35], [37], [38]

Binary Robust Independent Elementary Features (BRIEF)[39]

The BRIEF is a feature descriptor method that uses intensity difference to achieve high speed and efficiency. It operates by using a random selection process to determine a collection of location pairings inside a predetermined patch around a given key point. Subsequently, it proceeds to evaluate and contrast the intensity values associated with each pair. The BRIEF descriptor is generated by assigning a value of 1 to the relevant bit in the descriptor if the intensity in the first place is higher than the intensity in the second place. On the other hand, the bit is set to 0. BRIEF descriptors provide a high level of conciseness and demonstrate exceptional computational efficiency, making them very suitable for real-time applications.

The BRIEF descriptor is an approach for feature description that is known for its speed and efficiency, as well as its ability to maintain invariance to changes in illumination. The FAST corner detector[40] is often used in conjunction with it since it effectively identifies corner-like characteristics inside pictures. Although the BRIEF descriptor lacks invariance to rotation or scale,

The Oriented FAST and Rotated BRIEF (ORB) is a feature detector and descriptor that uses the FAST corner detector [40] and the BRIEF descriptor [39]. The property of being invariant to rotation and scale changes makes it very suitable for a diverse range of applications. The ORB does this by including an orientation estimation approach into the FAST detection algorithm and by selecting a predetermined set of intensity test pairings that exhibit little correlation.

Matching

Gradient-based feature descriptors, such as SIFT and SURF, have shown remarkable efficacy in the domain of picture matching. However, their effectiveness is accompanied by a significant computing cost due to their high dimensionality. In order to enhance the efficiency of matching speed, one may use tree-based methodologies like FLANN [41] or binary descriptors. These techniques provide expedited calculation of closest neighbors by using the least Hamming distance. The use of the Locality Sensitive Hashing (LSH)[42] technique enables a further reduction in the number

of characteristics that need evaluation. This reduction is achieved by only examining descriptors that possess identical subsets of descriptor bits. Efficient implementation of this may be achieved by the use of a randomized collection of hash tables.[35]

1.4.2 Neural Network

An artificial neural network (ANN) [43] is a mathematical model that draws inspiration from the structure and functioning of the human brain, allowing computers to learn and make decisions based on it. ANN consists of layers of linked neurons, with the weights of the connections influencing how information flows through the network. The fundamental components of artificial neural networks (ANN) consist of neurons. Neurons are interconnected by synapses, which serve as channels for the transmission of information.[44] The functioning mechanism of an artificial neuron is shown in Figure 1.2. The first artificial neural network architecture developed specifically for computer vision applications was the Neocognitron[45], which was presented by Fukushima in 1988. In 1989[46], LeCun became the first individual to effectively train a comparable neural network via the use of gradient descent and backpropagation techniques. Deep Learning is a computational approach that relies on the utilization of ANN [43] to address predominantly complex nonlinear difficulties

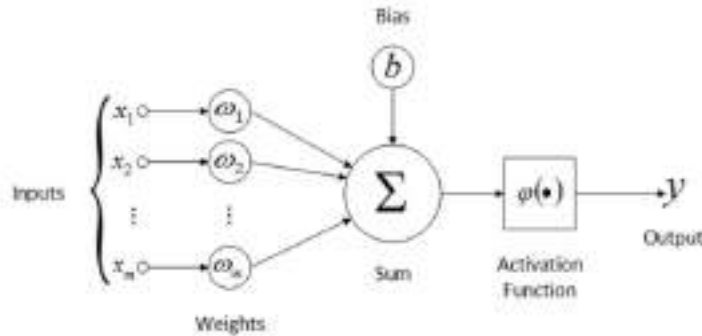


Figure 1.2: Principle of an artificial neuron [44]

Convolutional Neural Network (CNN)

The Convolutional Neural Network (CNN) is a type of artificial neural network. It has been shown to achieve superior levels of accuracy and, in several instances, enhanced efficiency when compared to conventional computer vision methodologies. [47] CNNs are comprised of a range of constituent elements, among which the three most prevalent are convolution layers, pooling layers, and fully connected layers. The convolutional and pooling layers play a crucial role in feature extraction, and the fully connected layers are responsible for mapping [48] these extracted characteristics to the ultimate output. In order to augment the contextual comprehension of a picture by a Convolutional Neural Network, two methodologies may be utilized: Conditional Random Fields (CRFs)[47] and Atrous Convolution[49].

Recurrent neural networks (RNNs)

The study of recurrent neural networks (RNNs) began throughout the 1980s and 1990s, eventually establishing them as a prominent deep learning method in the early 21st century. Long short-term memory networks (LSTMs) are a popular kind of RNNs that are extensively used. [50].

RNNs have the ability to transmit information from one state to the subsequent one, while simultaneously taking into account previously observed information. Nevertheless, early RNNs had the challenge of disappearing or exploding gradients, impeding their ability to adequately capture temporal information over long sequences. [51]

The ability of CNN to effectively collect temporal information, an essential component for the implementation of VO approaches based on end-to-end deep learning, is limited. In order to tackle this problem, the integration of RNNs with CNNs are used to estimate VO.

Deep recurrent neural networks (RCNNs) are a kind of neural network that integrates CNNs with RNNs in order to address the difficulties associated with disappearing or exploding gradients. RCNNs provide the concurrent acquisition of knowledge about representation and the sequential modeling of VO. This capability enables the efficient assimilation of temporal information.[2]

2 Literature Review

Firstly, the working of ORB-SLAM itself is investigated in Section 2.1. Then the working of the Deep VO is explained in section 2.2. The Architecture of deeplabv3+ is explained in Section 2.3.

2.1 ORB SLAM2

The ORB-SLAM [8], which was introduced in 2014, has since established itself as a widely respected framework within the domain of SLAM. The enhanced version of ORB-SLAM known as ORB-SLAM2,[1] is a feature-based SLAM system that has been optimized for better performance and it is compatible with monocular, stereo, and RGB-D cameras and was utilized as a baseline for evaluating most of the new slam-based systems.

The ORB-SLAM2 system consists of three primary concurrent threads, namely Tracking, Local Mapping, and Loop Closing. The primary objective of the Tracking thread is to estimate ego-motion, while the Local Mapping thread is responsible for building and optimizing the map using the local Bundle Adjustment (BA) technique. In contrast, the Loop Closing thread continuously tries to detect revisited situations using the use of pose-graph optimization. The above-mentioned threads work together in a collaborative manner to determine the camera's position, create a map, and correct issues related to drift and loop closures. The optimization approach used by ORB-SLAM2 makes use of the Levenberg-Marquardt algorithm implemented in the g2o [52] framework. The place recognition component involves the use of a bag of word [53] representation for the keyframes. The construction of the global map depends upon the covisibility graph,[54], [55] which is made up of nodes that represent keyframes and edges that establish connections between keyframes with common map points. To achieve loop closure, a basic graph is derived from the covisibility graph, consisting of the same number of nodes as the keyframes, but only including high co-visibility edges. The structure of the system is seen in Figure 2.1

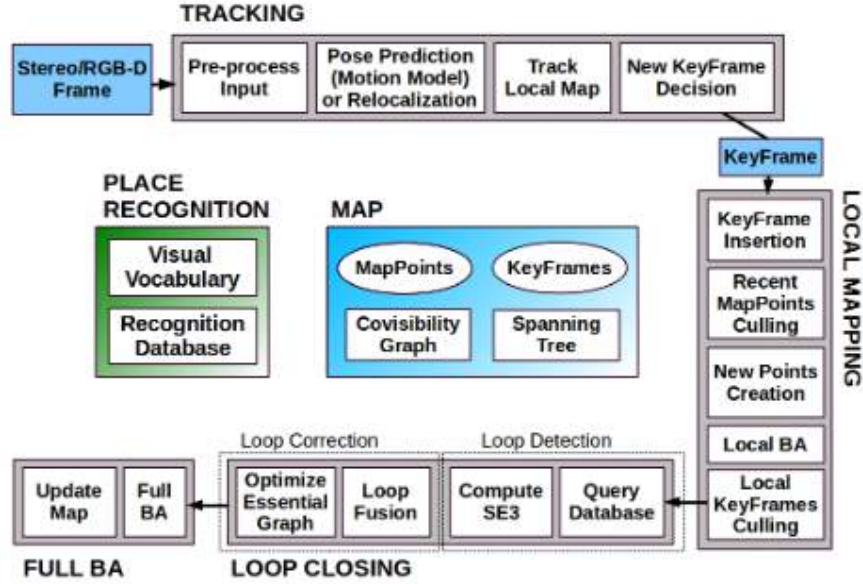


Figure 2.1: ORB SLAM2 System Threads and Modules [1]

2.1.1 Tracking

The tracking component of ORB-SLAM2 is responsible for estimating the camera posture of the current frame in relation to the previous frame, as well as making the decision of whether to generate a new keyframe. In the event of tracking loss, the place recognition module [56] is invoked with the objective of relocalizing the camera inside the global map.

Initially, the ORB feature points are matched between the current frame and the preceding frame. The optimization of the camera position for the current frame is performed via motion-only bundle adjustment (refer to Appendix of [8]). Subsequently, the identification of keyframes that exhibit overlap with the present frame is accomplished by the utilization of the co-visibility graph derived from preceding keyframes. The keyframes' coordinates are reprojected onto the current frame and then matched. The matches are used for the purpose of enhancing the estimation of the camera attitude. Ultimately, a decision is reached about the creation of a new keyframe.

The extraction of ORB points is performed on the current frame at eight different scale levels, with a scale factor of 1.2. The quantity of corners extracted is dependent upon the size of the image, wherein 1000 corners are used for dimensions ranging from 512x384 to 752x480, and 2000 corners

are utilised for bigger images. The scale levels have been divided into a grid, in which five FAST corners are extracted per cell, or more if required to get the desired amount of features. This results in the establishment of a homogeneous distribution of FAST corners throughout the frame. The FAST corners obtained are further characterized using the ORB descriptor.

After the extraction of ORB points from the current frame, their tracking may be achieved by using the camera's location in the previous frame. This process is accomplished by making the assumption that the camera maintains a constant velocity, and thereafter seeking to identify corresponding map points in the previous frame within a specific region around the predicted location of each ORB point. In the event that an insufficient number of corresponding map points are identified, the search zone is enlarged. After a sufficient number of matches have been identified, they are used to enhance the pose estimation of the present frame.

In situations when tracking is lost, a bag of words representation is created for the present frame. This representation is then used to search a database of keyframes in order to identify possible matches. After the identification of potential keyframes, the computation of ORB feature correspondences is performed between the current frame and each of the candidate keyframes. Subsequently, RANSAC(Random Sample Consensus) iterations are executed on each individual candidate keyframe in order to estimate a camera posture using the PnP (Perspective-n-Point method).[57] Once a camera pose is identified with a satisfactory amount of inliers, it undergoes optimisation. Subsequently, a guided search is conducted to identify more matches between the map points of the candidate keyframe and the camera position. Subsequently, the camera attitude is re-optimized, and the tracking method is continued if it is substantiated by a enough number of inliers.

The method furthermore maintains a local map in the form of a collection of keyframes. The set comprises the keyframes that share identical points with the current frame, together with all the neighboring keyframes. Every individual point inside the local map is transformed and projected into the current frame. After that, the algorithm verifies if the given point is situated inside the picture if its viewing direction from the current frame deviates by no more than 60 degrees from the average viewing direction of the point and if the distance between the camera centre and the point falls within the scale invariance range of the map point. When all of these requirements are satisfied, the point is compared with the unmatched ORB characteristics of the current frame in

order to identify more matches. Ultimately, a final optimisation is conducted by using all of the map points inside the frame.

ORB-SLAM employs the following criteria to determine the suitability of a frame for keyframe selection: In order to achieve a reliable relocalization, it is essential that the frame be positioned no less than 20 frames away from the most recent global relocalization. The local mapping thread is required to be in an idle state. In the event that a keyframe has not been inserted during the last 20 frames, the local mapping thread is halted to facilitate the insertion of the keyframe. Subsequently, the thread is restarted with the newly inserted keyframe. The frame should consist of a minimum of 50 points. Ensuring effective tracking is crucial. The number of corresponding points shared between a frame and its reference keyframe, which is the keyframe with the highest number of corresponding points, should not exceed 90% of the total number of points in the frame.

2.1.2 Map Initialization

The initialization of the map is based on the predicted position between the first two frames. The ORB-SLAM2 algorithm employs two distinct geometric models, each optimised for certain scene types. One model is designed to perform more effectively in planar environments, such as when the algorithm is observing a wall. The other model is tailored to excel in non-planar scenarios. An initial map is generated only if one of these two options delivers a sufficiently satisfactory configuration. The first phase involves acquiring the ORB matches, represented as points x , between the current frame and the reference frame. The preceding frame serves as the reference frame. If an insufficient number of matches are identified, the system will attempt another matching process with the subsequent frame. If a sufficient number of ORB matches are identified, the initialization process computes in parallel threads one based on homography and the other based on the fundamental matrix. The homography is calculated using the normalized Direct Linear Transform (DLT) method [58], using four points and iterating for a specified number of iterations. The computation of the basic matrix is performed with an eight-point method [58], employing 8 points for an equivalent number of iterations as the calculation of the homography.

Homography is used in situations when the scene exhibits planarity, near-planarity, or low parallax. Conversely, the fundamental matrix is used in situations where the scene lacks planarity and possesses sufficient parallax. Nevertheless, homography can be identified within a non-planar

scene under some circumstances. [59] This occurs when a subset of the matches either lie on a plane or exhibit minimal parallax. A heuristic is calculated to assert the appropriate methodology to use.

After the selection of a model, the motion hypotheses are retrieved. The homography model uses the Faugeras et al. [60] approach to obtain eight motion hypotheses, which are subsequently tested to chirality tests in order to identify the right solution. Nevertheless, the efficacy of these tests may be compromised in instances of minimal parallax, perhaps resulting in the incorrect identification of a solution. In order to deal with this matter, a total of eight solutions are triangulated to see which exhibits the highest number of spots observed by parallax, positioned in front of both cameras, and has a minimal reprojection error. In the absence of a solution, the initialization phase is not executed and commences from the initial step. The utilization of this strategy improves the resilience of the initialization process in scenarios with minimal parallax.

In the context of the fundamental matrix model, the initial step involves the conversion of the fundamental matrix into an essential matrix through the utilization of the calibration matrix and the four motion hypotheses are obtained utilizing the singular value decomposition technique. Subsequently, the four solutions are subjected to triangulation, and the reconstruction is chosen based on identical criteria as for the homography model. In order to enhance the accuracy of the initial reconstruction, a full bundle adjustment BA (see the Appendix [8]) is conducted.

2.1.3 Local Mapping

KeyFrame Insertion

The first step involves the modification of the covisibility graph, when a new node is introduced. The new node is then connected to all other keyframes with the shared map points. Next, we proceed with updating the spanning tree that connects the new node to the keyframe that has the highest number of map points with it. Finally, the bag of words representation is computed, helping the association of new points with pre-existing keyframes during the process of triangulation.

Recent Map Points Culling

A map point refers to a specific location inside the surroundings that has been captured and recorded by the camera on multiple occasions. To be sure that a map point is real and not just a mistake, it has to pass a test. The test criteria are: the map point should be visible in over 25% of the three keyframes after it is created. If this condition is met, then the map point is genuine and is retained. After passing the test, a map point may only be eliminated if it is missing from at least three frames. This may occur when keyframes are discarded by the local bundle adjustment. This policy makes sure that there aren't many fake map points, or outliers, on the map.

New Map Point Creation

The generation of new map points in ORB-SLAM involves the process of triangulating ORB features derived from connected keyframes inside the covisibility graph. It conducts a search for matches between mismatched ORB features in a keyframe and unmatched features in another keyframe while excluding matches that fail to meet the epipolar constraint. After identifying a pair of corresponding ORB features, the system proceeds to perform triangulation in order to obtain a new map point. The acceptance criteria for a new map point include the presence of positive depth in both cameras, a significant parallax between the two keyframes, a reprojection error that falls below a predetermined threshold, and a scale that aligns with the entire image. At the beginning, a map point is first observed in two keyframes, but other keyframes also exhibit the potential for matching the same map point. Consequently, the algorithm proceeds to project the map point onto the remaining related keyframes and starts a search for correspondences.

The local BA procedure focuses on optimizing a specific subset of the keyframes and map points inside the local map. The subset comprises the keyframes that are linked to the present keyframe inside the covisibility graph, along with all the map points that are observed by these keyframes. The optimization process includes additional keyframes that observe the same map points as the current keyframe but are not directly linked to it. These additional keyframes, although included in the optimization, are kept fixed and do not undergo any adjustments. During the optimization process, outlier observations are removed.

2.1.4 Loop Closure

The loop closure process in ORB-SLAM2 consists of two distinct stages: loop identification and validation, which are then followed by pose-graph optimization. The process of loop identification relies on the comparison of features present in the current frame with a collection of keyframes stored in the map database. The validation of loops is conducted by using geometric restrictions and leveraging stereo/depth information in order to determine the validity of the identified loop. After a loop has been confirmed, the system proceeds to execute pose-graph optimization in order to rectify the accumulated drift that has occurred since the loop closure.

In addition to pose-graph optimization, ORB-SLAM2 integrates a complete bundle adjustment optimization process to get the most optimum solution. The optimization process is executed in a separate thread to enable the system to concurrently generate maps and identify loops. However, this presents the difficulty of integrating the result of bundle modification with the existing state of the map. In the event of new loop is identified during the execution of the optimization process, the optimization procedure is terminated and the loop is closed, initiating a Full BA optimization again.

Upon completion of the whole Bundle Adjustment (BA) process, the updated subset of keyframes and points is integrated with the original keyframes and points that were not altered during the optimization phase. This is done by spreading the correction of updated keyframes to non-updated keyframes via the spanning tree. Non-updated points are transformed accordingly.

2.2 Deep VO

Traditional methods for VO often rely on pipelines that involve feature extraction, recognition, and tracking. These methods can be limited in texture-less environments or when the images are noisy. Despite the significant effectiveness exhibited by Deep Learning methodologies in tackling various computer vision challenges, their application in perception-related tasks within autonomous navigation. DeepVO is an end-to-end VO system that combines deep learning (DL) techniques with recurrent neural networks (RNNs)[61] The Deep VO method is capable of directly estimating camera pose by analyzing raw images, eliminating the requirement of feature extraction or matching. This feature enables the utilization of a greater amount of information contained within the image, hence enhancing the accuracy and robustness of our technique, particularly in surroundings lacking distinct textures. The main contribution is threefold

- 1 Deep learning has the potential to address the monocular visual odometry problem through an end-to-end approach. DeepVO is a first-of-its-kind methodology for monocular visual odometry VO utilizing deep neural networks (DNNs). This approach allows for the direct estimation of positions from raw RGB images, without the need of parameters (camera calibration).

- 2) Proposed a new RCNN-based architecture that enables DL-based visual odometry algorithms to generalize to entirely new environments by using the geometric feature representations learned by CNN.

- 3) Deep Recurrent Neural Networks (RNNs) can implicitly capture and learn the complex motion dynamics and sequential dependence of an image sequence, these characteristics are crucial for visual odometry VO, however challenging for humans to explicitly model.

2.2.1 Architecture of the Proposed RCNN

The utilization of geometric feature representations holds significant importance in the field of visual odometry and other related geometric challenges. VO systems are designed to process image sequences that are captured while objects are in motion and construction of models that establish links between consecutive image frames. The RCNN (Recurrent Convolutional Neural Network) under consideration addresses both of these needs. The architecture of the proposed Deep VO system is shown in Figure 2.2.

The VO system takes in a monocular image sequence as input and preprocesses the input image to normalize it. Then, two consecutive images are combined together to form a tensor, which is fed into the CNN. The CNN extracts feature from the image tensor, which are then passed to the RNN for sequential learning. The RNN learns to model the temporal dynamics of the VO problem and outputs a pose estimate for the camera at each time step. The advantage of the RCNN-based architecture is that it allows simultaneous feature extraction along with sequential modeling of VO. This means that the system can learn to extract features that are useful for VO and to model the temporal dynamics of the problem in a single step. This makes the system more efficient and accurate than traditional VO systems, which typically perform feature extraction and sequential modeling separately.

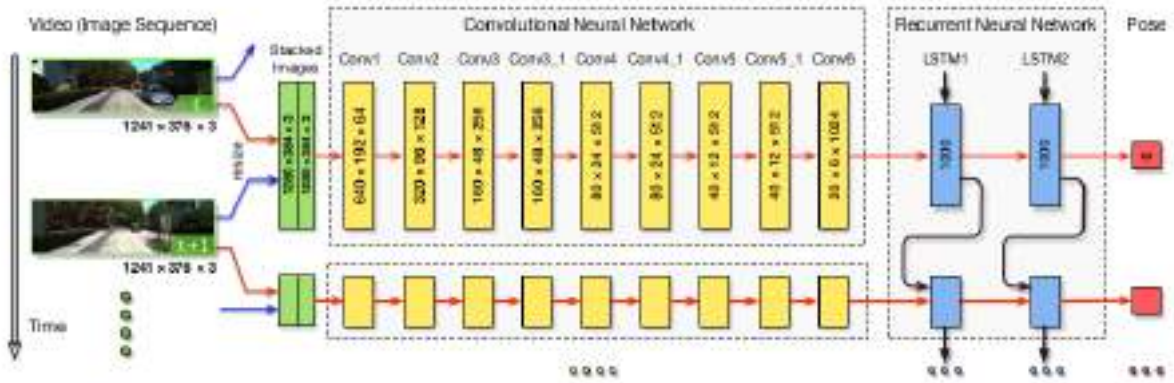


Figure 2.2: Architecture of the proposed RCNN-based monocular VO system [2]

2.2.2 CNN-based Feature Extraction

A Convolutional Neural Network (CNN) has been developed with the purpose of extracting effective features from consecutive monocular RGB images to address the VO challenge. The selection of features is primarily focused on geometric attributes rather than appearance or visual context. This approach enables the VO to effectively adapt and perform in unexpected environments. The architecture of CNN draws inspiration from the network specifically created for the purpose of estimating optical flow.[62] The configuration of the CNN is outlined on the KITTI dataset is shown in Figure 2.3

The CNN architecture for visual odometry consists of 17 layers, including 9 convolutional

Layer	Receptive Field Size	Padding	Stride	Number of Channels
Conv1	7×7	3	2	64
Conv2	5×5	2	2	128
Conv3	5×5	2	2	256
Conv3_1	3×3	1	1	256
Conv4	3×3	1	2	512
Conv4_1	3×3	1	1	512
Conv5	3×3	1	2	512
Conv5_1	3×3	1	1	512
Conv6	3×3	1	2	1024

Figure 2.3: Configuring of the CNN [2]

layers. The convolutional layers use filters of different sizes in order to extract characteristics from the input image. The dimensions of the filters gradually decreased in order to capture more features. zero-padding is used in order to maintain the original spatial dimensions of the image following each convolution process. The number of filters has been increased in order to get a diverse range of features. The CNN uses unprocessed RGB images as its input, rather than using pre-processed images like optical flow or depth images. It occurs because of the network is trained to acquire its own optimized feature representation for visual odometry. The acquired feature representation compresses the initial RGB picture with high dimensionality into a compact description to enhance the future sequential training process. The last convolutional feature, Conv6, is then sent to the RNN to facilitate sequential modeling. The RNN learns to represent the sequential changes in the characteristics, hence facilitating the estimation of the camera's position through a given time period.

2.2.3 RNN-based Sequential Modelling

Sequential learning is performed using a deep RNN following the use of a CNN. This implies that the recurrent neural network is capable of capturing dynamics and interdependencies between a sequence of CNN features. The RNN does this task implicitly, using relevant sequential information. The Recurrent Neural Network is a suitable choice for addressing the Visual Odometry issue due to its ability to capture sequential dependencies. This characteristic is crucial for calculating the position of the current image frame by using information from previous frames.

However, RNNs are not feasible for directly learning sequential representations from images. In order to address this constraint, the system uses a recurrent convolutional neural network (RCNN) structure, where the convolutional neural network features are used as the input to the recurrent neural network. The Recurrent Neural Network exhibits distinct characteristics when compared to the Convolutional Neural Network. One key distinction is the RNN's ability to retain a memory of its hidden states over a sequence of inputs, hence enabling the incorporation of feedback loops among these states. This enables the incorporation of the previous hidden states as a variable in the current state, hence enabling the recurrent neural network to identify correlations between the input and the previous states within the sequence. The RNN can learn to predict the current pose of the camera by taking into account the previous poses and the current image features, making it a powerful tool for visual odometry. Therefore, the Recurrent Neural Network is capable of identifying the relationships between the input and the preceding states within a given sequence. At time step k , a recurrent neural network updates the convolutional feature x_k .

Given a convolutional feature x_k at time k , a Recurrent Neural Network updates at time step k by

$$h_k = H(W_x h x_k + W_h h h_{k-1} + b_h) \quad (2.1)$$

$$y_k = W_y h_k + b_y \quad (2.2)$$

In these equations, h_k and y_k are the hidden state and output at time k , respectively. The W terms denote corresponding weight matrices, the b terms denote bias vectors, and H is an element-wise non-linear activation function, such as sigmoid or hyperbolic tangent.

The proposed system employs a Long LSTM network to determine the relationships between consecutive images. By employing memory gates and units, LSTM networks are able to learn long-

term dependencies. These memory gates and units control explicitly which previous concealed states are discarded or retained for revising the current state, enabling the LSTM to learn motion during pose estimation.

The proposed system employs an LSTM network to determine the relationships between consecutive images. By employing memory gates and units, LSTM networks are able to learn long-term dependencies. These memory gates and units control explicitly which previous concealed states are discarded or retained for revising the current state, enabling the LSTM to learn motion during pose estimation. The unfolded version of the LSTM network demonstrates how each LSTM unit is associated with a time phase, allowing the network to accurately model the sequential nature of the input data. Figure 2.4 illustrates the folded LSTM and its unfolded representation over a period of time, along with the internal architecture of an LSTM unit.

Given the input x_k at time k and the hidden state h_{k-1} and the memory cell c_{k-1} of the previous Long Short-Term Memory unit, the LSTM updates at time step k according to [2]

$$i_k = \sigma(W_{xi}x_k + W_{hi}h_{k-1} + b_i) \quad (2.3)$$

$$f_k = \sigma(W_{xf}x_k + W_{hf}h_{k-1} + b_f) \quad (2.4)$$

$$g_k = \tanh(W_{xg}x_k + W_{hg}h_{k-1} + b_g) \quad (2.5)$$

$$c_k = f_k c_{k-1} + i_k g_k \quad (2.6)$$

$$o_k = \sigma(W_{xo}x_k + W_{ho}h_{k-1} + b_o) \quad (2.7)$$

$$h_k = o_k \tanh(c_k) \quad (2.8)$$

In these equations, σ denotes the sigmoid non-linearity, $\tanh$ is the hyperbolic tangent non-linearity, the W terms represent corresponding weight matrices, the b terms represent vectors, i_k , f_k , g_k , c_k , and o_k are the input gate, forget gate, input modulation gate, memory cell, and output gate at time k , respectively.

The deep RNN architecture is composed of a sequential arrangement of two LSTM layers on top of each other. The hidden states of the first LSTM layer are used as input to the second LSTM layer. This allows the network to learn more complex representations of the data. The deep RNN produces a pose estimation at each time step by using the visual features generated from a CNN. This means that the network is able to track the pose of a person or object over time as it moves.

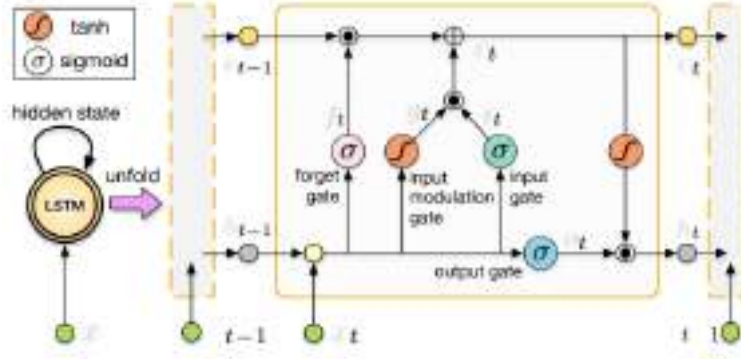


Figure 2.4: Folded and unfolded LSTMs and internal structure of its unit. [2]

2.2.4 Cost Function and Optimization

The system based on RCNN can be regarded as a method for calculating the conditional probability of the poses. Given a probabilistic perspective, let $Y_t = (y_1, \dots, y_t)$ represent a succession of monocular RGB images $X_t = (x_1, \dots, x_t)$ up to time t in the probabilistic perspective:

$$p(Y_t|X_t) = p(y_1, \dots, y_t|x_1, \dots, x_t) \quad (2.9)$$

In the deep RCNN, the processes of modeling and probabilistic inference are executed. In order to choose the most suitable parameters θ^* for the variational optimization (VO) technique, the DNN aims to maximize a certain objective function. (2.9):

$$\theta^* = \arg \max_{\theta} p(Y_t|X_t; \theta) \quad (2.10)$$

In order to figure out the hyperparameters θ of the Deep Neural Networks (DNNs), the objective is to minimize the Euclidean distance between the ground truth posture (p_k, ϕ_k) at time k and its estimated pose $(\hat{p}_k, \hat{\phi}_k)$. The loss function consists of the Mean Square Error (MSE) calculated for all points p and orientation. ϕ :

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^t \|\hat{p}_k - p_k\|_2^2 + \kappa \|\hat{\phi}_k - \phi_k\|_2^2 \quad (2.11)$$

The 2-norm, denoted as $\|\cdot\|_2$, is used in this context. The scale factor κ , which has a value of 100 in the experiments, is employed to balance the weights of positions and orientations. Additionally, N represents the number of samples. The orientation ϕ is denoted by Euler angles.

2.3 Semantic Segmentation

Semantic segmentation is a computer vision task whereby the objective is to assign a class label to each individual pixel in an image, therefore generating a segmentation map. This process is sometimes referred to as pixel-level classification since it entails the grouping of picture components that pertain to a certain class of objects exhibiting comparable characteristics.[63]

The dominant approach for image segmentation in the present day is the use of artificial intelligence and deep neural networks. The efficacy of deep neural networks in picture segmentation has been shown to be very effective, surpassing what was previously deemed unattainable only a decade ago .[64]

Semantic segmentation involves the use of a model that accepts an input picture and generates outputs in the form of areas together with their corresponding class labels. The process involves converting the pixel values of the input picture into corresponding class label values and generates a segmentation map, which serves as a comprehensive depiction of the picture, whereby every individual pixel is allocated to a distinct class. [47]The development of semantic segmentation network designs occurred together with the advancement of object detection Object detection networks are designed to identify and categorize certain regions of interest within an image. In contrast, semantic segmentation networks have the objective of categorizing every individual pixel inside the picture.

2.3.1 DeepLab

DeepLab [65] is a cutting-edge Deep Convolutional Neural Network (DCNN) used for semantic picture segmentation, which has undergone consistent development and enhancement since 2014. [65], [66] Image segmentation has found applications in several domains, including the analysis of medical pictures for tumor detection and the identification of pedestrians in photos. DeepLab network has undergone iterative enhancements, wherein each subsequent version has integrated novel features and approaches. The DeepLabv2[67] model incorporated the Atrous Spatial Pyramid Pooling (ASPP) technique, enabling the network to effectively capture contextual information at several scales. DeepLabv3 [68] has eliminated the necessity of employing Conditional Random Fields (CRFs) for post-processing, thereby transforming it into a comprehensive learning platform. DeepLabv3+ represents the most recent iteration of the DeepLab network, exhibiting superior per-

formance compared to its predecessors. DeepLabV3+, [3] which has been created by Google, is the most advanced and cutting-edge model for semantic segmentation within the DeepLab series. This has become a baseline model for numerous applications.

2.3.2 DeepLabV3+

The DeepLabV3+ [3] model is an advanced semantic segmentation architecture that builds upon the DeepLabV3 model. It does this by including an encoder-decoder structure and using the Xception backbone. The encoder uses atrous convolution as a means to extract dense feature maps, while maintaining a low output stride of 16. The inclusion of semantic segmentation is of significance as it enables the model to effectively capture intricate information. The decoder component within the architecture is tasked with enhancing the accuracy of the segmentation process by refining the bounds of objects. This is achieved by combining the upsampled features from the encoder with the low-level features from the backbone via concatenation. Subsequently, the outcome is subjected to a 3x3 convolution operation to enhance its quality, followed by an upsampling process that increases its dimensions by a factor of four. The Deeplabv3+ network is shown in th Figure 2.5.

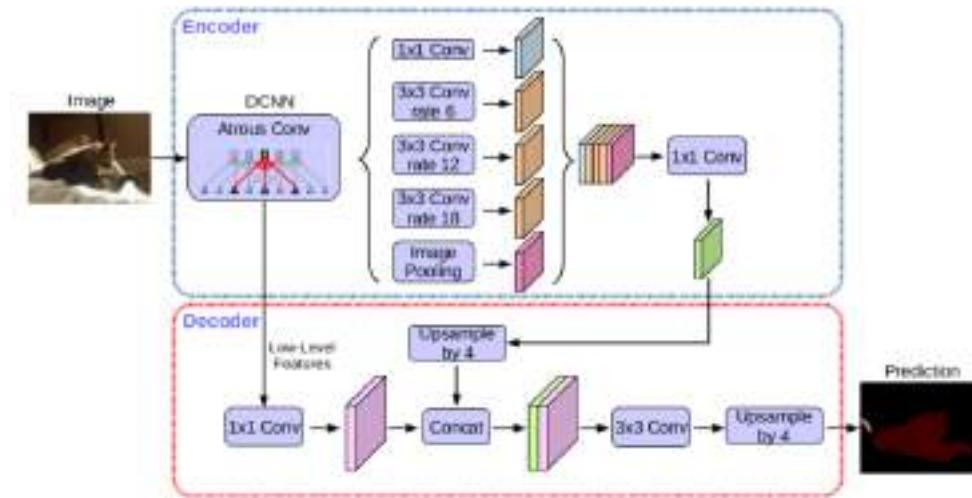


Figure 2.5: Deeplabv3+ network [3]

3 Research Methodology

This section will discuss the experimental setup for ORBSLAM2, including the parameters that were utilized in Section 3.1.1. Following that, Section 3.1.2 will discuss the testing model employed for DeepVO, together with the necessary libraries. Section 3.1.3 provides an overview of the configuration and architecture of the deeplabv3+ model, as well as a detailed explanation of the setup. Section 3.2 will discuss the dataset used for the evaluation of odometry algorithms. Section 3.3 of the paper focuses on the theoretical aspects pertaining to quantitative metrics, namely Absolute Trajectory Error(ATE) and Relative Pose Error (RPE). These measures are commonly employed to assess and compare different odometry approaches.

3.1 Experimental setup

The experiments are conducted on the Lenovo Ideapad laptop Intel Core i7 12700H processor, 16 GB of RAM, and an NVIDIA RTX 3060 graphics card running Ubuntu 20.04.6 LTS with ROS Neotic

3.1.1 ORB-SLAM2

The Robot Operating System (ROS) facilitates the clear visualization of the specific data that is exchanged between nodes, hence providing a valuable tool for debugging purposes. The exchange of information across ROS nodes is facilitated by the use of topics. The first procedure included the installation of ROS and ORB-SLAM2 [69] on the laptop. The image sequences are retrieved from a specified directory and sent through a topic to ORB SLAM 2. In order to facilitate visualization, the RViz and rqt tools were used, both of which are 3D visualization.

Parameters

Camera.fx: 707.0912 ; Camera.fy: 707.0912; Camera.cx: 601.8873 ; Camera.cy: 183.1104

3.1.2 Deep VO

Libraries and Software Components

The DeepVO SLAM algorithm can be implemented using the following libraries and software components: Clear steps and running of the code are described in [70]

- PyTorch 0.4.0
- TorchVision 0.2.1
- NumPy
- Pandas
- Pillow
- Matplotlib
- Glob
- Trained model
- Optimizer

Training and Testing:

The DeepVO model is trained on Sequences 00, 01, 02,05, 08and 09 and then tested on Sequences 04, 05, 06, 07,

The trained model can be downloaded from the following link: Download DeepVO Pretrained Model.[71]

Steps to work with DEEP VO

1. The code utilized for the purposes of testing and training has been obtained from the GitHub repository [70].

2. To obtain the KITTI dataset, the user can utilize the KITTI/downloader.sh script provided in the downloaded code and the trained mode from [71].
3. To eliminate unused photos in the KITTI devkit, the user should execute the 'preprocess.py' script as instructed in the accompanying readme file. [70]
4. Execute the main.py file in order to initiate the training process of the model.
5. Execute the test.py script in order to generate the anticipated poses as output.
6. Execute the visualize.py script. Execute the visualize.py script to observe the prediction made by the DeepVO model.

3.1.3 Semantic Segmentation

For the semantic segmentation deeplabv3+ [3] model has been used. The pre-trained model has been trained on the city space dataset and can be downloaded from [72]. The code for the model testing can be downloaded from [73] The semantic output of DeepLab masks dynamic items, such as buses, trucks, people, and motorcycles, by representing them in black color. The semantic image is then analyzed for black pixels, These pixels are masked and utilized to modify the original image, to create a masked image.

The DeepLabv3+ semantic segmentation model is executed by a Python script(see Appendix) in order to generate the initial mask. After acquiring the segmentation mask, it is subsequently altered to generate a mask covering only the dynamic items in the frame. The changed mask is then published on a topic. The Orb SLAM [1] algorithm utilizes masked images as input and executes operations on these masked images. The system accepts the masked frame in its typical manner, but, it excludes the ORB features of the masked region during the process of feature extraction. The findings section presents the orb slam features seen in both the masked and unmasked photos. The process in orb slam takes in masked images as inputs and the system is termed Masked ORBSLAM

2.

3.1.4 Simple VO

The development of a simple Visual Odometry (VO) system is completely using OpenCV libraries. The trajectory of a moving camera is estimated by calculating the camera positions between consecutive frames and accumulating them. The code Appendix Listing A.1: uses ORB features to extract distinctive features from a pair of successive images, which are subsequently matched using the Brute-Force Matcher (BFMatcher). The matches are thereafter subjected to filtration based on the distance threshold. After applying the filtering process to the matches, the code proceeds to estimate the essential matrix (E) by utilising the matched features and the camera intrinsic matrix (K). By utilising the essential matrix, the algorithm proceeds to estimate the rotation matrix (R) and translation vector (T) that describe the geometric transformation between the two images. This is utilised for estimating the pose of the object within the given scene. In order to approximate the path of the object, it computes the cumulative relative displacement between successive frames and the resulting trajectory is saved in a file.

- `cv2.ORB_create()` function is a feature detector and descriptor that exhibits robustness and efficiency while also maintaining invariance to variations in light and rotation.
- `cv2.detectAndCompute()` function is utilized to identify and calculate ORB features from an image. The function outputs a collection of keypoints and a NumPy array that stores the descriptors corresponding to each keypoint.
- `cv2.BFMatcher()` function is used for matching features between two images by performing a thorough comparison of the descriptors associated with each pair of features.
- `cv2.knnMatch()` function is used to establish feature correspondences between two images by utilizing the k-nearest neighbors approach. The function provides a collection of k-nearest neighbor matches for every feature.
- `cv2.findEssentialMat()` function is utilized to compute the Essential Matrix, which represents the intrinsic relationship between two images. The Essential Matrix is a square matrix with dimensions 3×3 that establishes a relationship between the camera poses of two given images. The utilization of this technique enables the estimation of the rotation matrix and translation vector that correspond to the transformation between the two images.

- `cv2.recoverPose()` function is utilized to estimate the rotation matrix and translation vector that correspond to the transformation between two images, based on the Essential Matrix. Additionally, the function has the capability to generate an inlier mask. This mask serves the purpose of indicating the specific features that were utilized during the estimation process of the rotation matrix and translation vector.

3.2 Dataset

The KITTI dataset[4] is extensively used as a helpful tool for assessing the effectiveness of autonomous driving algorithms. It provides an extensive collection of authentic data obtained from diverse traffic situations. The dataset was produced by driving through several traffic circumstances in Karlsruhe, Germany. The collection comprises sets of stereo greyscale and color images, ground truth LIDAR data, and high-accuracy GPS location measurements. The KITTI[4] dataset is a great resource for academics and developers engaged in the advancement of autonomous driving technologies.

In the context of testing masked ORBSLAM [1], could not find a dataset that has moving objects in the images along with calibration and ground truth information. A new dataset was generated by taking few images from the the KITTI [4] dataset sequence to form new sequences to evaluate the performance of masked ORBSLAM in the presence of moving objects.

3.2.1 Modified Sequence

Sequence 1 is an altered version of the KITTI [4] dataset sequence 07, encompassing images numbered from 610 to 770. The composition encompasses an intersection point and dynamic entities, a truck that enters the visual field and traverses alongside other vehicles.

Sequence 2: The dataset images 0 to 600 from the KITTI [4] dataset have been subjected to alteration. It has few cars moving in the scene highway scenario.

Sequence 3 The sequence is third sequence from the sample sequence from the ApolloScape dataset. [74]

3.3 Error Metrics

3.3.1 Absolute translation error (ATE)

Absolute translation error (ATE) is the measure of the distance difference between the predicted trajectory from the odometry method to the ground truth trajectory in three-dimensional space. In our possession, we possess:

$$T_i := \sqrt{(GT_{i_x} - P_{i_x})^2 + (GT_{i_y} - P_{i_y})^2 + (GT_{i_z} - P_{i_z})^2}$$

where GT_x , GT_y , and GT_z are the x, y, and z translation components of the ground truth pose, and P_x , P_y , and P_z are the x, y, and z translation components of the estimated pose from visual odometry. Then, we will compute the RMSE value:

$$RMSE(T_{1:n}) := \left(\frac{1}{n} \sum_{i=1}^n T_i^2 \right)^{1/2}$$

3.3.2 Relative pose error (RPE)

The relative pose error (RPE) measures the local accuracy of the trajectory over a fixed time interval Δ , corresponding to the drift in the trajectory. The relative pose error at time step i is defined as:

$$E_i := (Q_{i-1}Q_{i+\Delta})^{-1}(P_{i-1}P_{i+\Delta})$$

In this way, we obtain $m = n - \Delta$ individual relative pose errors from a sequence of n camera poses. From these errors, we will compute the root mean squared error (RMSE) of the translational component:

$$RMSE(E_{1:n,\Delta}) := \left(\frac{1}{m} \sum_{i=1}^m \|\text{trans}(E_i)\|^2 \right)^{1/2}$$

where $\text{trans}(E_i)$ is the translational component of the relative pose error E_i .

The KITTI dataset has data recorded at 10Hz, which means that for RPE calculation, Δ will be set to 10, thus $RMSE(RPE)$ represents the effective drift per second.

4 Results

Section 4.1, presents a comparative analysis of the trajectories and errors exhibited by the Orb SLAM2 and DeepVO algorithms in relation to the ground truth. The trajectory plot for the overall sequences in detail and plotted the results of this analysis, including Absolute Trajectory Error (ATE) and Relative Pose Error (RPE), in a bar graph for both algorithms. The units for ATE and the translation part of RPE are both in meters.

Section 4.2 provides a statistical analysis of dynamic objects that were discovered by DeepLabV3+ on sequences 04, 05, 06 and 07 of the KITTI dataset[4]. The analysis is presented in the form of bar graphs for visualisation purposes. The efficacy of DeepLabV3+ is comprehensively assessed, showcasing its proficiency in detecting and segmenting moving objects within the surrounding environment. An image of masked image obstacle used by the ORB-SLAM2 algorithm is also included. Additionally, this section presents trajectory plots of ORB-SLAM and masked ORB-SLAM for modified sequences 3.2.1 providing a visual comparison of their performance in these specific scenarios.

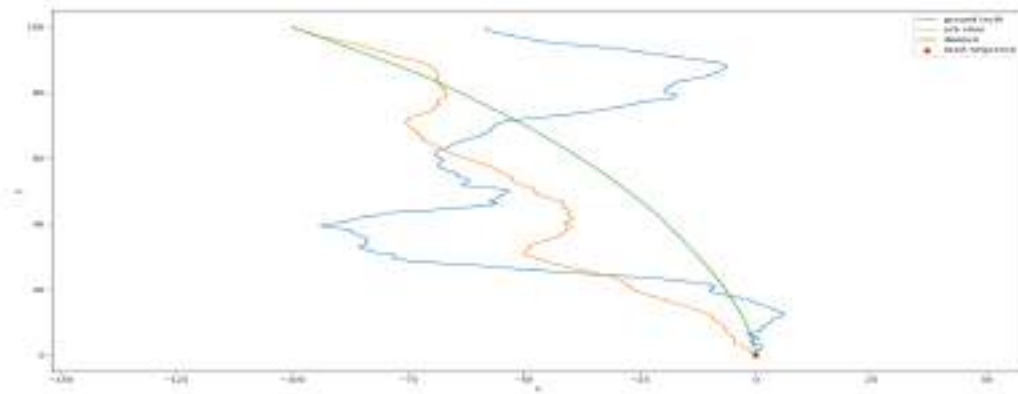
In section 4.3, the results of the simple visual odometry 4.3 method, along with the ORB-SLAM2 and DeepVO algorithms, on the KITTI dataset[4] are presented. The results section provides a trajectory plot analysis to evaluate the performance of the algorithms, for the sequence 04, 05, 09 of kitti [4] dataset.

4.1 Comparison of Orbslam and Deepvo

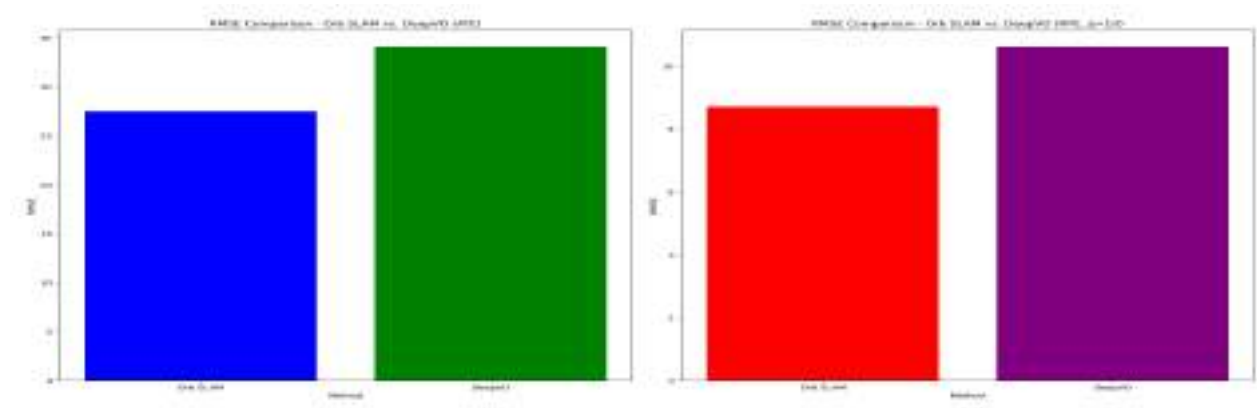
The DeepVO model was trained exclusively on a specific subset of the Kitti dataset[4], namely Sequences 00, 01, 02, 05, and 08. To evaluate the model's performance, it was tested on Sequences 04, 05, 06, and 07. The outcomes of the model's trajectories and errors for the sequences and comparison with ORB SLAM2 are shown below in this section.

4.1.1 Sequence 04

In the study ORB-SLAM2 and DeepVO algorithms are evaluated on KITTI[4] Sequence 04. The findings are shown in a trajectory chart and error as bar plot. Figure a shows ORB-SLAM2 (orange) and DeepVO (green) trajectory charts. A ground truth trajectory is shown in blue. ORB-SLAM2 fits ground truth trajectory better than DeepVO. ATE and RPE error charts are shown in Figure b for ORB-SLAM2 and DeepVO. ORB-SLAM2 has 27.50m ATE, while DeepVO had 34.09m. The RPE of ORB-SLAM2 was 10.62m and DeepVO 8.72m. ATE and RPE were lower in ORB-SLAM2 than DeepVO.



(a) ORBSLAM2 and DeepVO trajectories compared to ground truth on sequence 4

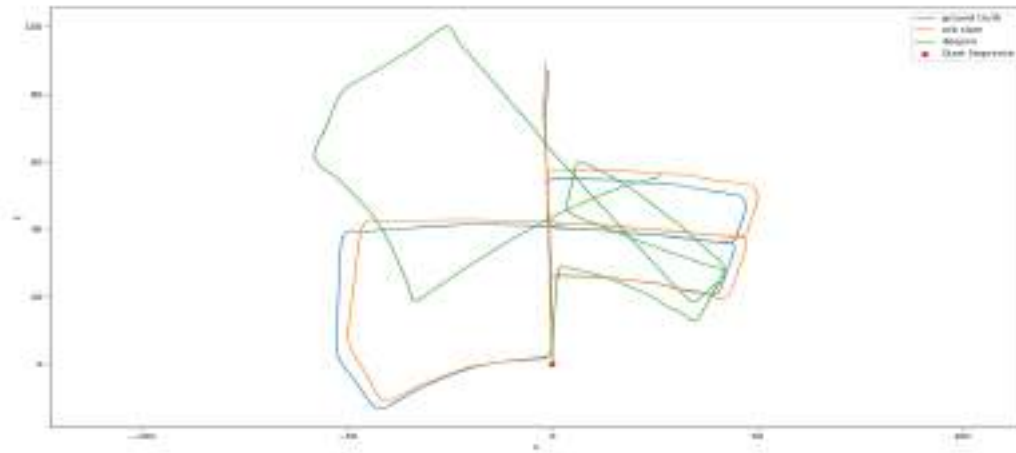


(b) ATE and RPE for ORB-SLAM and DeepVO on sequence 4

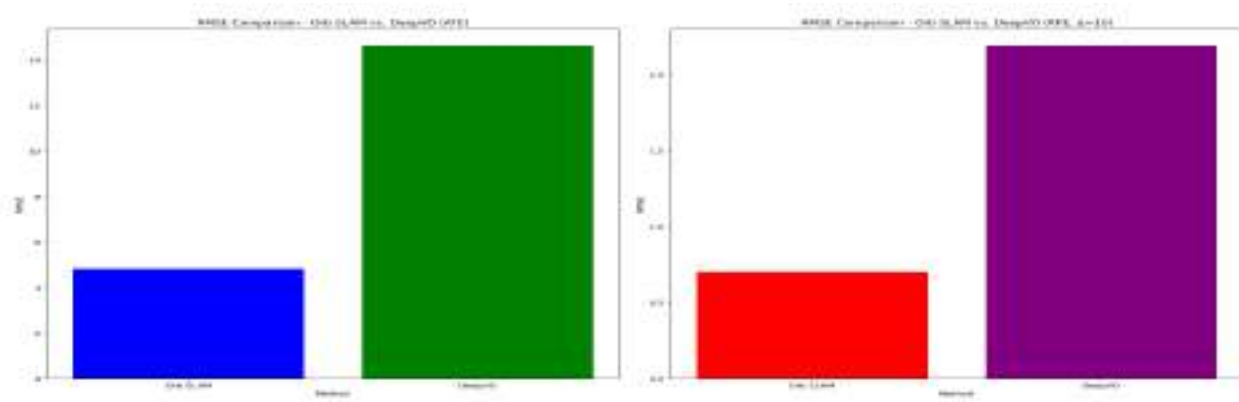
Figure 4.1: Trajectory and Error plots for KITTI [4] sequence 4

4.1.2 Sequence 05

ORB-SLAM2 and DeepVO algorithms are tested on KITTI[4] Sequence 05. The findings are shown in a trajectory chart and error plot. Figure a shows ORB-SLAM2 (orange) and DeepVO (green) trajectory charts. A ground truth trajectory is shown in blue. ORB-SLAM2 fits ground truth trajectory better than DeepVO. ATE and RPE error charts are shown in Figure b. ORB-SLAM has 4.834 m ATE, while DeepVO had 14.622 m. The RPE of ORB-SLAM2 was 0.704m and DeepVO 2.19m. ATE and RPE were lower in ORB-SLAM2 than DeepVO.



(a) ORBSLAM2 and DeepVO trajectories compared to ground truth on sequence 5

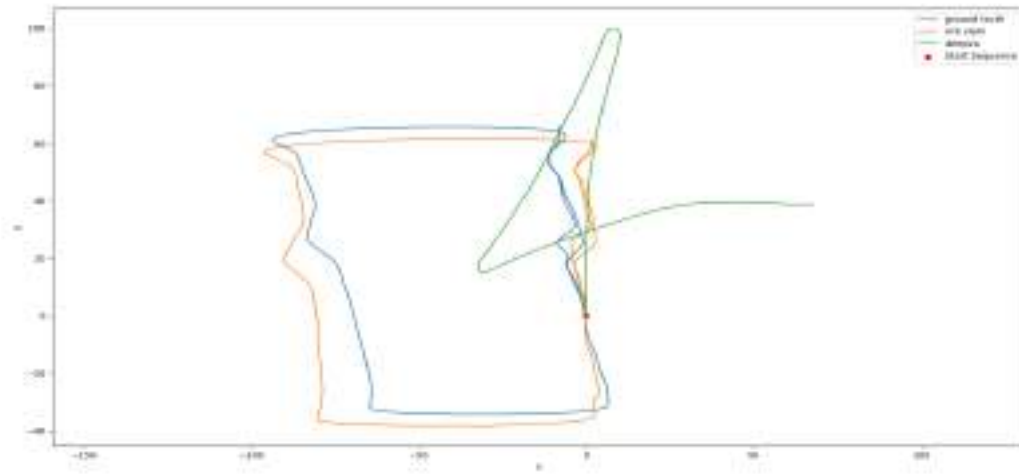


(b) ATE and RPE for ORB-SLAM and DeepVO on sequence 5

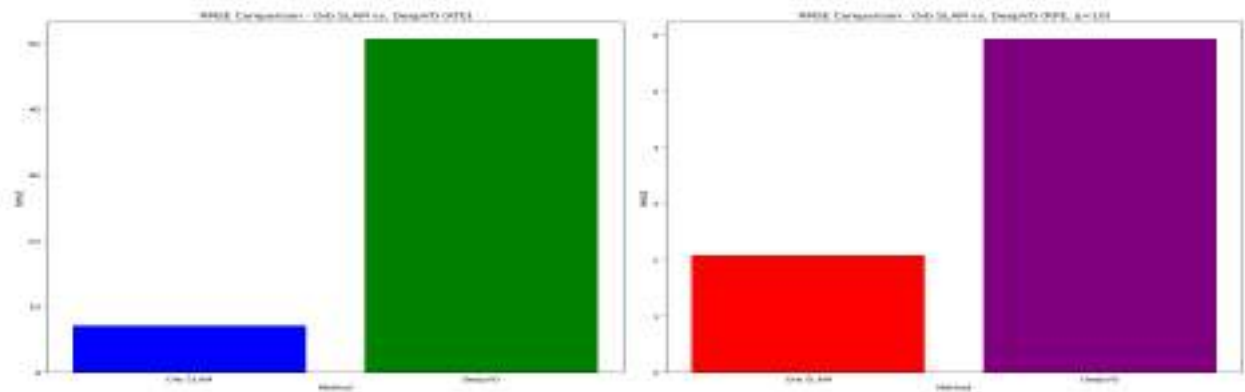
Figure 4.2: Trajectory and Error plots for KITTI [4] sequence 5

4.1.3 Sequence 06

ORB-SLAM2 and DeepVO algorithms are tested on KITTI[4] Sequence 06. Figure a shows ORB-SLAM2 (orange) and DeepVO (green) trajectory charts. A ground truth trajectory is shown in blue. ORB-SLAM2 fits ground truth trajectory better than DeepVO. ATE and RPE error charts are shown in Figure b for ORB-SLAM2 and DeepVO. ORB-SLAM2 has 7.13 m ATE, while DeepVO had 50.73 m. The RPE of ORB-SLAM2 was 2.08 m and DeepVO 5.93m . ATE and RPE were lower in ORB-SLAM2 than DeepVO.



(a) ORBSLAM2 and DeepVO trajectories compared to ground truth on sequence 6

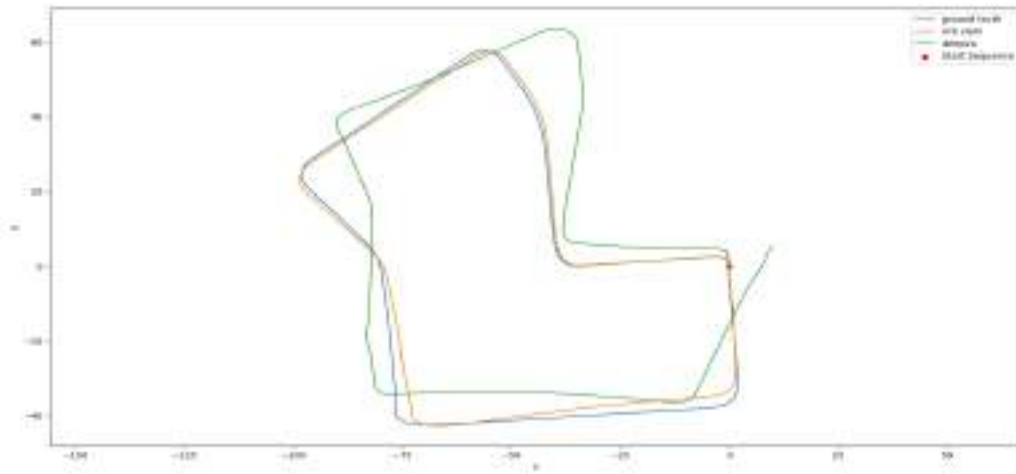


(b) ATE and RPE for ORB-SLAM and DeepVO on sequence 6

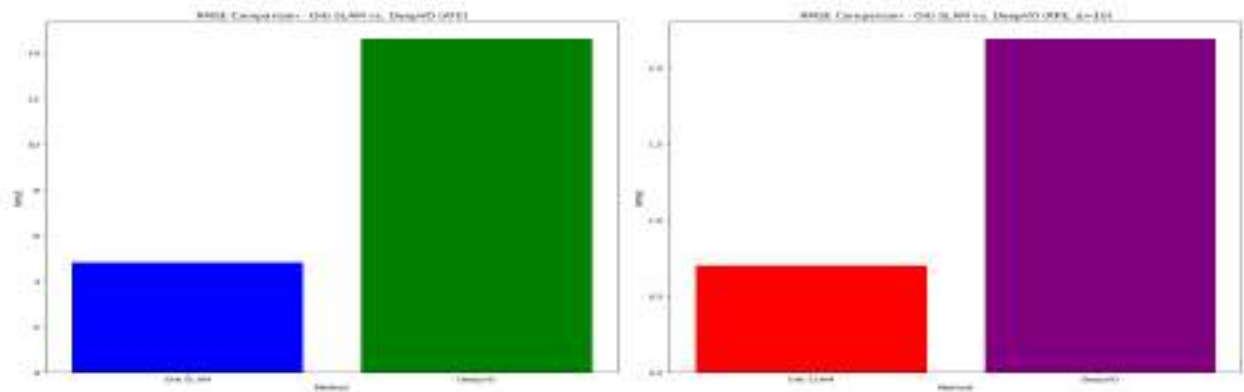
Figure 4.3: Trajectory and Error plots for KITTI [4] sequence 6

4.1.4 Sequence 07

ORB-SLAM2 and DeepVO algorithms are tested on KITTI[4] Sequence 07. . Figure a shows ORB-SLAM2 (orange) and DeepVO (green) trajectory charts. A ground truth trajectory is shown in blue. ORB-SLAM2 fits ground truth trajectory better than DeepVO. ATE and RPE error charts are shown in Figure b for ORB-SLAM2 and DeepVO. ORB-SLAM2 has 2.755m ATE, while DeepVO had 44.37m. The RPE of ORB-SLAM2 was 1.09m and DeepVO 4.46m. ATE and RPE were lower in ORB-SLAM2 than DeepVO.



(a) ORBSLAM2 vs DeepVO trajectories compared to ground truth on sequence 7



(b) ATE and RPE for ORB-SLAM and DeepVO on sequence 7

Figure 4.4: Trajectory and Error plots for KITTI [4] sequence 7

Sequence number	ATE ORB SLAM2	ATE DeepVO	RPE ORB SLAM2	RPE DeepVO
Sequence 04	27.50	34.09	10.62	8.72
Sequence 05	4.834	14.622	0.704	2.19
Sequence 06	7.13	50.73	2.08	5.93
Sequence 07	2.755	44.37	1.09	4.46

Table 4.1: ORBSLAM and DeepVO Error Comparision in Meters.

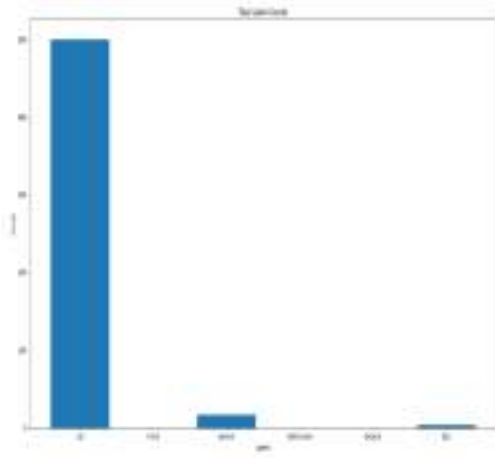
The test conducted on ORB-SLAM2 and DeepVO indicates that ORBSLAM2 shows higher accuracy in relation to Absolute Trajectory Error and Relative Pose Error throughout all four sequences that were examined. The trajectory plots clearly demonstrate that ORBSLAM2 generates trajectories that are smoother and more consistent compared to Deepvo with ground truth . However, it is important to note that the performance of these algorithms may vary depending on the specific dataset and conditions.

4.2 Semantic Segmentation and Masked ORBSLAM2

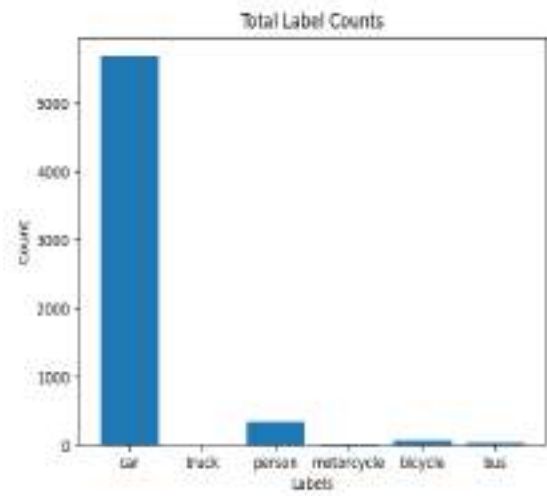
Section 4.2.1 provides statistical information regarding the quantity of dynamic entities, including buses, trucks, pedestrians, and motorbikes, that have been observed inside a certain sequence. The section 4.2.2 demonstrates the semantic output generated by DeepLabv3+ when applied to an image sourced from the Kitti dataset[4]. Furthermore, Section 4.2.4 presents a demonstration of the ORB characteristics on both the mask picture and the original image. In Section 4.2.5 presents a comparative analysis of the ORB-SLAM2 and the masked ORB-SLAM2.

4.2.1 Statistical data

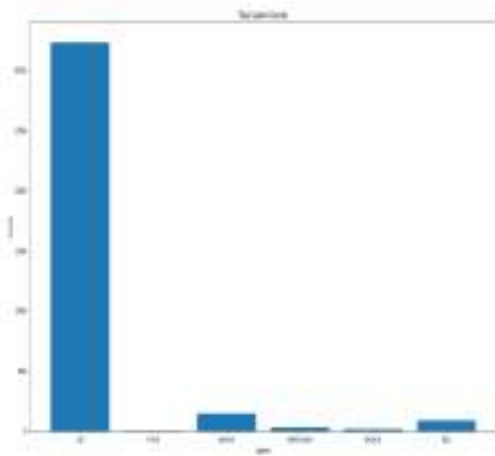
The existence of dynamic obstacles presents a significant problem for VSLAM. The statistical data shows the number of objects in the sequence . The information can be used as valuable information for VSLAM systems in dynamic environments. However, removing these obstacles from the input data can improve the performance and robustness of VSLAM systems.



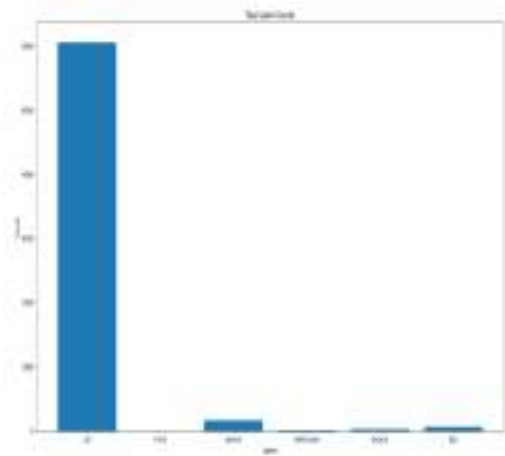
(a) Statistical data for sequence 04



(b) Statistical data for sequence 05



(c) Statistical data for sequence 06



(d) Statistical data for sequence 07

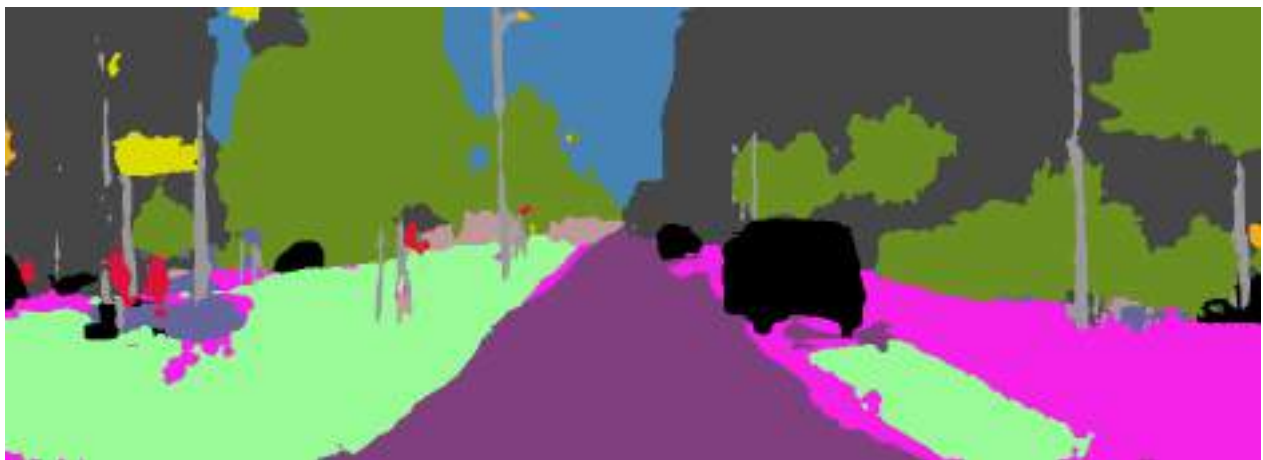
Figure 4.5: Statistical data on Kitti [4] Sequences

4.2.2 Deeplab V3+ semantic segmentation output

This section shows an analysis of the visual performance exhibited by the DeepLabv3+ model when applied to an image sourced from the Kitti dataset[4] sequence 06. Given the absence of ground truth labels the evaluation of the model's performance is limited to visual assessment. The DeepLabv3+ model successfully produced a semantic segmentation result that effectively detected and characterised prominent elements within the image, such as the road, sidewalk, cars, and trees.



(a) Original image from Kitti dataset



(b) Semantic Image for the original image

Figure 4.6: Original and Semantic output from Deeplabv3+[3]

4.2.3 Masked output

Masking of dynamic objects from the input frame, that will be utilised by Masked ORB-SLAM2. An original and masked image of a dynamic obstacle from the KITTI dataset[4] is presented to illustrate this hypothesis. Visual inspection confirms that the study's results on the KITTI dataset [4] present encouraging qualitative outcomes, thereby providing support for the hypothesis.



(a) Original Image



(b) Masked Image

Figure 4.7: Original and masked image

4.2.4 ORB Feature

The objective of this study is to examine the effects of masking on ORB feature extraction from ORB-SLAM, using both original and masked images obtained from the KITTI dataset[4]. The hypothesis is that ORB-SLAM would not perform feature extraction on the masked part of the image. The findings of this study provide visual evidence in favour of our initial hypothesis. ORB-SLAM would extract a significantly less number of features from images that have been masked compared to unaltered original images.



(a) ORB Features on normal image



(b) ORB Features on masked image

Figure 4.8: Comparison of ORB Features on masked and original

4.2.5 Masked ORB SLAM2

This section showcases the trajectory plots for the modified sequences 3.2.1 output from masked ORB-SLAM2 and ORB-SLAM2.

Sequence 1

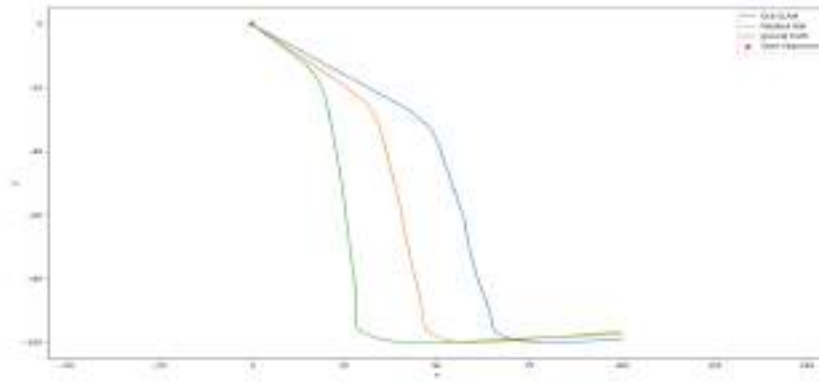


Figure 4.9: Comparison of masked ORBSLAM on modified sequence 01

Sequence 2

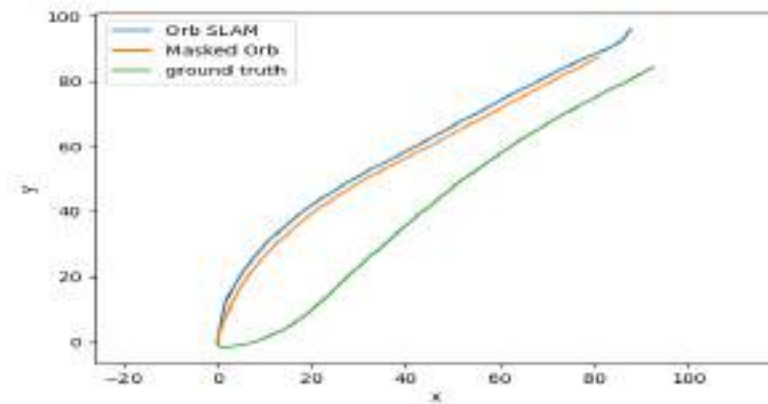


Figure 4.10: Comparison of masked ORBSLAM on modified sequence 02

Sequence 3

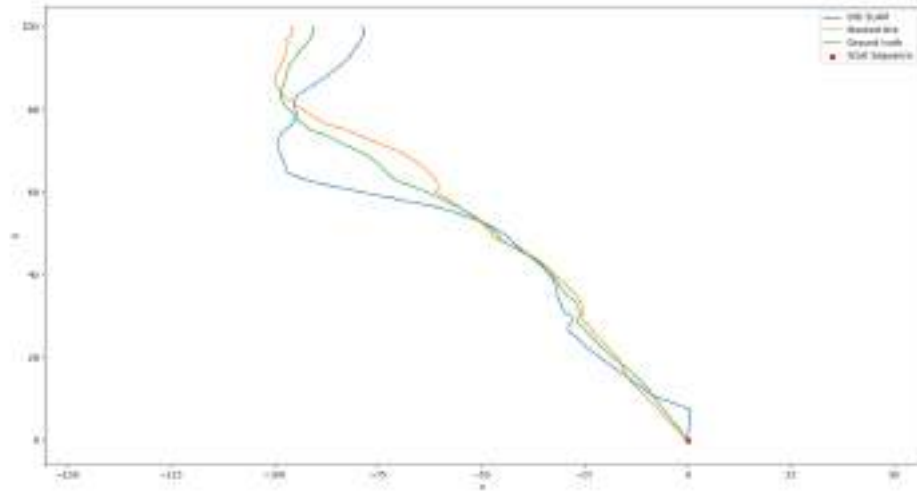
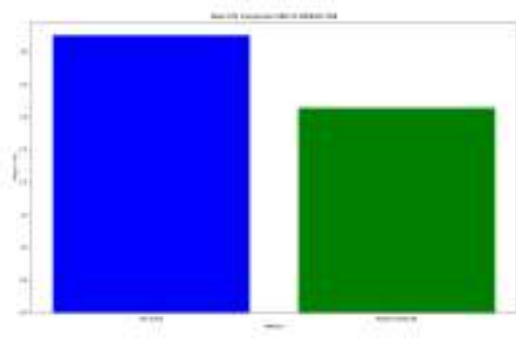
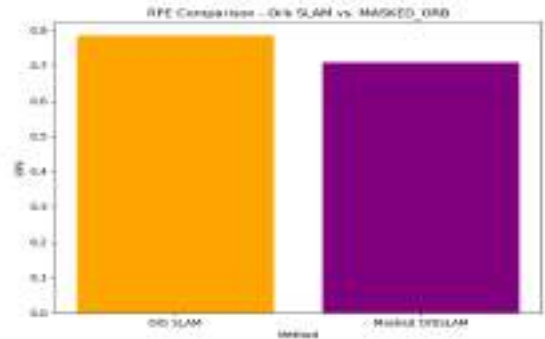


Figure 4.11: ORBSLAM2 and Masked ORBSLAM2 on modified sequence 03



(a) ATE

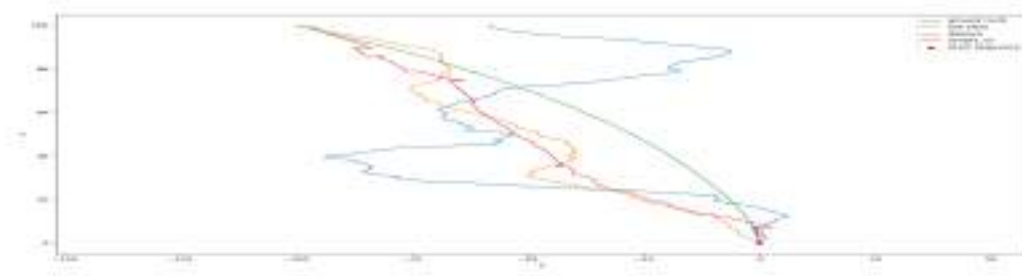


(b) RPE

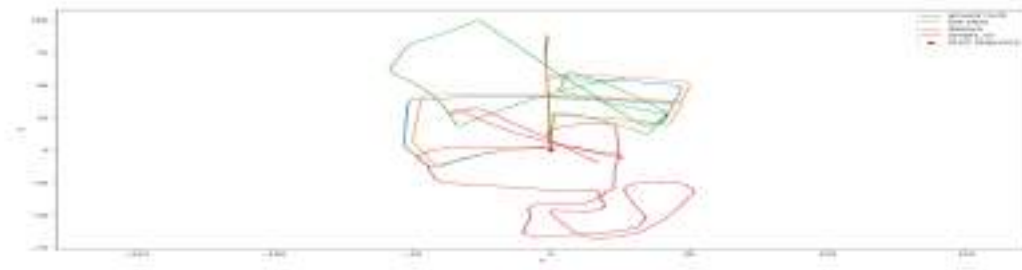
Figure 4.12: Error comparison of ORB SLAM2 and Masked ORB SLAM2 on ApolloSpace dataset

4.3 Comparison with simple VO

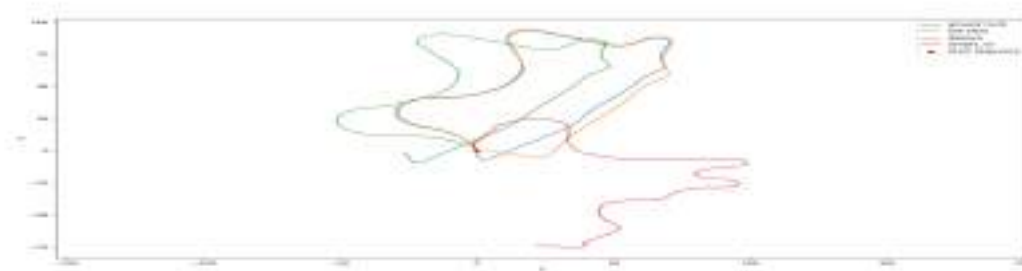
The present investigation aimed to evaluate the efficacy of three different visual odometry techniques, specifically the simple VO 3.1.4 method, ORB-SLAM2 [1], and DeepVO [2], in relation to their performance on kitti sequences 04, 05, and 09. The findings indicated that the simple visual odometry approach did not exhibit better performance compared to ORB-SLAM2 [1] DeepVO[2]. The RPE on sequence 05 of the simple VO approach was 1.0 meters, while that of ORB-SLAM2 and DeepVO were 0.1 meters and 0.5 meters.



(a) Sequence 04



(b) Sequence 05



(c) Sequence 09

Figure 4.13: Comparison of simple VO with ORBSLAM2 and DEEPVO on KITTI dataset [4]

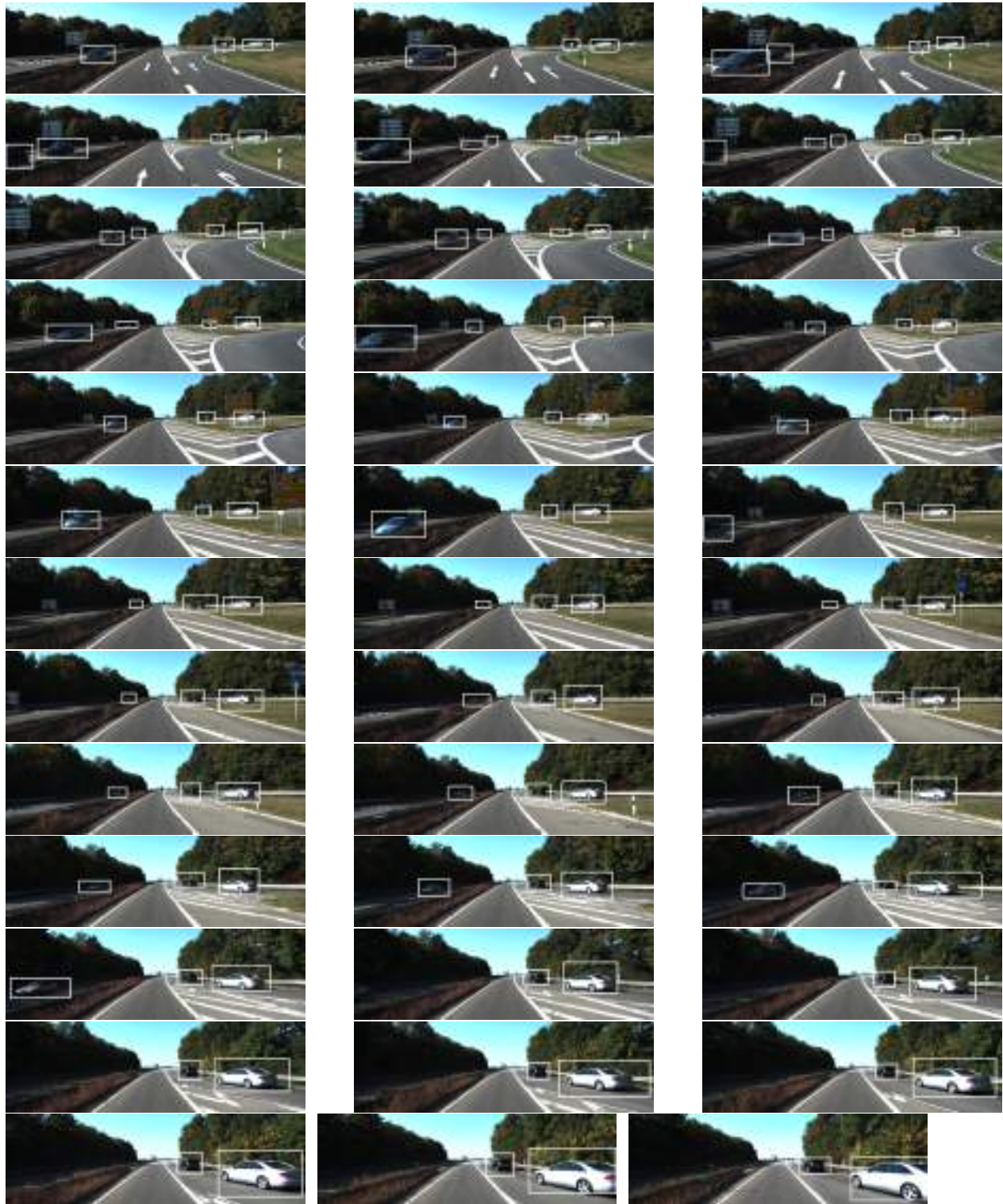


Figure 4.14: Images for modified Sequence 02

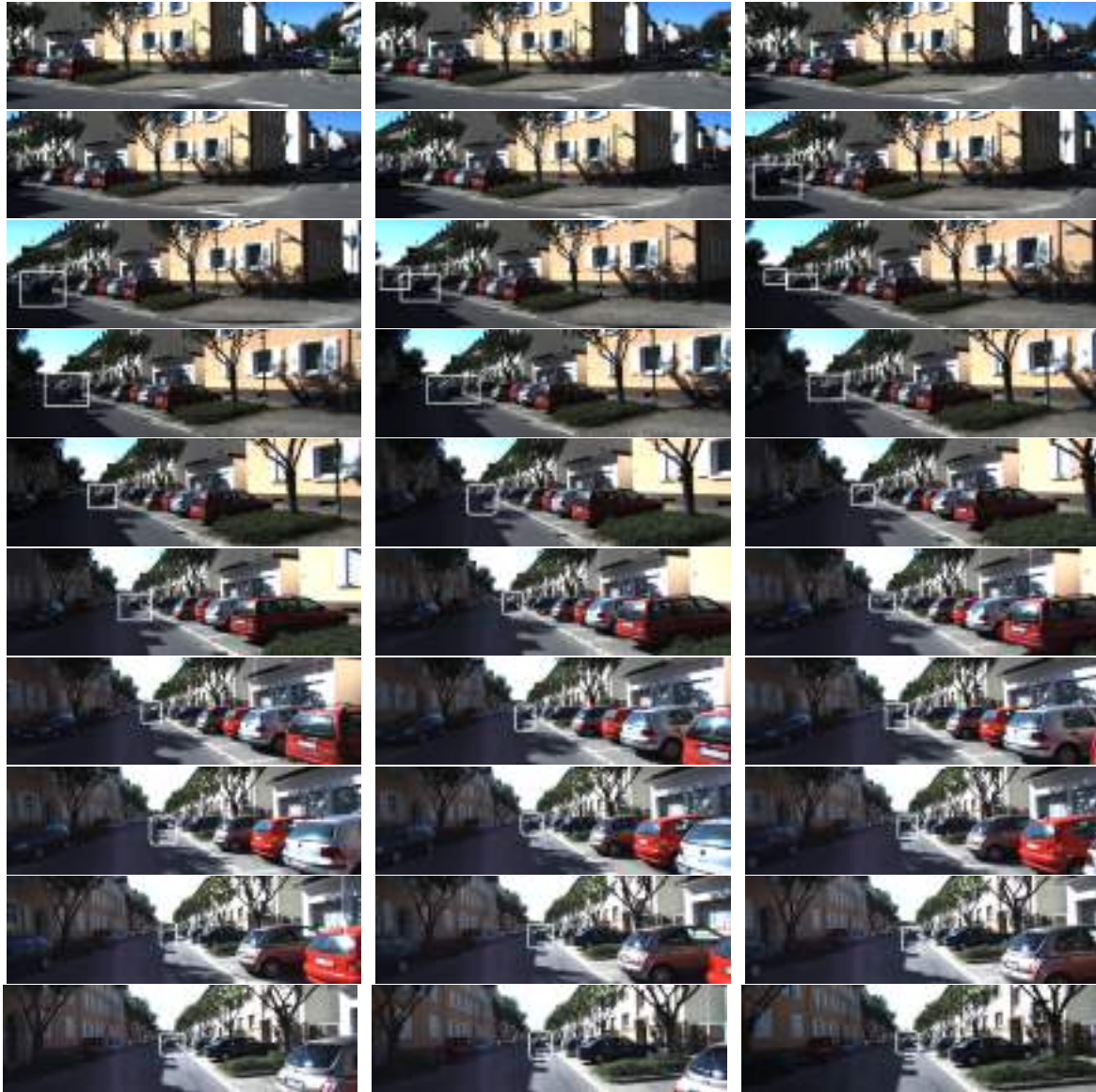


Figure 4.15: Few Images for modified Sequence 01



Figure 4.16: Images for apollo space dataset

5 Discussion and Conclusion

The evaluation performed in the previous chapter has indicated that Traditional SLAM methods tend to exhibit better results compared to deep learning-based SLAM approaches.

The evaluation also shows that the masking of dynamic obstacles might increase the performance of the VSLAM pipeline.

The efficiency of a simple visual odometry (VO) system is constrained by the lack of loop closure to facilitate position optimization and efficient feature extraction, filtering, and tracking methods.

5.0.1 ORB SLAM2 and Deepvo

The deep learning-based slam model, DeepVO, could not yield satisfactory outcomes in comparison to the conventional SLAM approach, ORBSLAM2.

Limitation

- DL-based SLAM methods are a newer area of research which needs improvement, while traditional SLAM methods have undergone substantial development and refinement.
- The assessment was performed using the KITTI dataset, which is a specialized dataset for outdoor environmental variations. Consequently, the outcomes might not have generalization to all potential cases.
- The testing has not been done with different other SLAM techniques .

Impact

This study offers significant contributions to understanding the efficacy of various SLAM method for outdoor data, hence serving as a helpful reference for informing future investigations in this domain.

Future work

- Evaluate the performance on different datasets with different environmental conditions.
- Train the DeepVO algorithm with different type of datasets. Training DeepVO on datasets with diverse scenes can improve the model's ability to generalize to new environments and predict motion in a wider range of conditions
- Investigate the potential of Inertial Measurement Unit (IMU) data to provide insights into camera motion and orientation.
- Explore ways to effectively incorporate IMU data into deep learning-driven SLAM techniques, such as DeepVO, to enhance precision and resilience.

5.0.2 Semantic Segmentation and Masked ORBSLAM

The potential enhancement of performance in the ORB-SLAM2 algorithm can be achieved by the incorporation of a masking technique. However, the effectiveness of the approach has not been extensively investigated in more challenging scenarios, such as situations with heavy traffic or the presence of moving objects. The lack of publicly available datasets that include camera calibration and ground truth for these specific settings is the underlying cause of this issue.

While the Apollo space dataset [74] and KAIST dataset [75] are available, it is worth noting that the ORB-SLAM system encounters difficulties in initializing or maintaining tracking of the original dataset, namely the non-masked images.

Limitation

- The approach has not been evaluated for variations in dataset size, variations in data quality, and variations in lighting conditions.
- The deeplab v3+ model demonstrates a significant latency in its inference process. The optimization of computing efficiency is of the utmost importance in the context of real-world applications.

- The deeplabv3+ model has undergone training using the city space dataset, and it has demonstrated satisfactory performance when applied to the Kitti dataset.[4] However, when applied to the Kaist dataset,[75] the generated masks exhibit suboptimal segmentation quality.
- In an environment where most of the objects (objects detected by the semantic segmentation like cars) in the image are static, masked ORBSLAM slam would mask all the objects which would affect the efficiency of the output.

Impact

- Research has the potential to bring about a significant transformation in SLAM techniques within dynamic environments. The advancement in this field would allow robots to navigate and execute tasks with a high degree of safety and autonomy in complex and dynamic surroundings.
- This research would provide valuable insights into the issues associated with working in dynamic environments and develop effective strategies to tackle them.

Future work

- In order to enhance the system's performance, investigate the integration of semantic segmentation along with optical flow techniques to enable identification of potentially dynamic objects and facilitate the selective excluding of only those parts within the image.
- Enhancing the segmentation performance of a deep learning model across several datasets can be achieved by training it on a more extensive dataset.
- Another area for investigation is improving the performance of the system when ORBSLAM2 is not working because it has not been initialized or has lost track for different datasets.

In summary, this study was a good exercise in the significance of integrating semantic information with VSLAM. By understanding the beneficial combination of these two technologies, to create more efficient and effective SLAM systems for various applications, to operate in complex and dynamic environments.

A Appendix

```
1 import cv2
2 import numpy as np
3 import os
4
5 def extract_features(image):
6     orb = cv2.ORB_create()
7     kp, des = orb.detectAndCompute(image, None)
8     return kp, des
9
10 def match_features(des1, des2):
11     bf = cv2.BFMatcher()
12     match = bf.knnMatch(des1, des2, k=2)
13     return match
14
15
16 def filter_matches(matches, kp1, kp2):
17     pts1 = []
18     pts2 = []
19     for i, (m, n) in enumerate(matches):
20         pts1.append(kp1[m.queryIdx].pt)
21         pts2.append(kp2[m.trainIdx].pt)
22     pts1 = np.float32(pts1)
23     pts2 = np.float32(pts2)
24     F, mask = cv2.findFundamentalMat(pts1, pts2, cv2.FM_RANSAC)
25     pts1 = pts1[mask.ravel()==1]
```

```

26     pts2 = pts2[mask.ravel()==1]
27     return pts1, pts2
28
29 def estimate_motion(pts1, pts2, K):
30     E, mask = cv2.findEssentialMat(pts1, pts2, K)
31     retval, R, t, mask = cv2.recoverPose(E, pts1, pts2, K)
32     return R, t
33
34 def estimate_trajectory(image_paths, K, trajectory_file=None):
35     trajectory = [np.array([0, 0, 0])]
36     RT = np.eye(4)
37
38     for i in range(len(image_paths) - 1):
39         image1 = cv2.imread(image_paths[i], cv2.
40                               IMREAD_GRAYSCALE)
41
42         image2 = cv2.imread(image_paths[i + 1], cv2.
43                               IMREAD_GRAYSCALE)
44
45         kp1, des1 = extract_features(image1)
46         kp2, des2 = extract_features(image2)
47
48         matches = match_features(des1, des2)
49         pts1, pts2 = filter_matches(matches, kp1, kp2)
50
51         R, t = estimate_motion(pts1, pts2, K)
52
53         rt_mt = np.hstack([R, t])
54         rt_mt = np.vstack([rt_mt, np.zeros([1, 4])])
55         rt_mt[-1, -1] = 1

```

```

54     rt_mt_inv = np.linalg.inv(rt_mt)
55     R_T = np.dot(RT, rt_mt_inv)
56     new_trajectory = R_T[:3, 3]
57     trajectory.append(new_trajectory)
58
59     print("Current Trajectory:")
60     print(new_trajectory)
61     if trajectory_file is not None:
62         with open(trajectory_file, "a") as file:
63             file.write(f"{new_trajectory[0]}, {
64                 new_trajectory[1]}, {new_trajectory[2]}\n")
65     trajectory = np.array(trajectory).T
66     return trajectory
67 image_folder = "/home/tarun/Documents/diss_deepvo/DeepVO-
68     pytorch/DeepVO-pytorch-master/KITTI/images/01"
69 image_files = sorted([os.path.join(image_folder, f) for f in os
70     .listdir(image_folder) if f.endswith('.png')])
71 K = np.array([[707.0912, 0, 601.8873], [0, 707.0912, 183.1104],
72     [0, 0, 1]])
73 trajectory_file = "trajectory.txt"
74 resulting_trajectory = estimate_trajectory(image_files, K,
75     trajectory_file)

```

Listing A.1: Python code for simple visual odometry

```

1 #!/usr/bin/env python3
2 from pyexpat import model
3 import network
4 import utils
5 import os
6 import numpy as np
7 from datasets import Cityscapes
8 from torchvision import transforms as T
9 import torch
10 import torch.nn as nn
11 from PIL import Image as im
12 import sys
13 import cv2
14 print(' - cv2.__file__ = ',cv2.__file__)
15 import rospy
16 from sensor_msgs.msg import Image as rosImage
17 from cv_bridge import CvBridge, CvBridgeError
18 from PIL import Image
19 global count
20 count = 0
21
22
23 def load_model():
24     global model, device
25     device = torch.device('cuda' if torch.cuda.is_available()
26                           else 'cpu')
27     model = network.deeplabv3plus_mobilenet(num_classes=19,
28                                             output_stride=16)
29     utils.set_bn_momentum(model.backbone, momentum=0.01)

```

```

28 ckpt = "/home/tarun/Documents/deeplab/model/
    best_deeplabv3plus_mobilenet_cityscapes_os16.pth"
29 if os.path.isfile(ckpt):
30     checkpoint = torch.load(ckpt, map_location=torch.device
        ('cpu'))
31     model.load_state_dict(checkpoint["model_state"])
32     model = nn.DataParallel(model)
33     model.to(device)
34     print("Resume model from %s" % ckpt)
35     del checkpoint
36 else:
37     print("[!] Retrain")
38     model = nn.DataParallel(model)
39     model.to(device)
40 model = model.eval()
41
42
43
44 def imageCallback(image_msg):
45     global device, count
46     cv_img = CvBridge().imgmsg_to_cv2(image_msg, "bgr8")
47     pil_img = im.fromarray(cv2.cvtColor(cv_img, cv2.
        COLOR_BGR2RGB))
48     #print("Image size for pil_img :", pil_img.size)
49     image_pub = rospy.Publisher("/semantic/image", rosImage,
        queue_size=10)
50     image_pub2 = rospy.Publisher("/masked/image", rosImage,
        queue_size=10)
51     transform = T.Compose([
52         T.ToTensor(),

```

```

53         T.Normalize(mean=[0.485, 0.456, 0.406],
54                      std=[0.229, 0.224, 0.225]),
55     ])
56     with torch.no_grad():
57         pil_img = transform(pil_img).unsqueeze(0) # To tensor
58         # of NCHW
59         pil_img = pil_img.to(device)
60
61         pred = model(pil_img).max(1)[1].cpu().numpy()[0] # HW
62         #print("size of pred:", pred.size)
63         decode_fn = Cityscapes.decode_target
64         colored_preds = decode_fn(pred).astype('uint8')
65         #print("size of colored_preds:", colored_preds.size)
66
67         colored_preds = im.fromarray(colored_preds)
68         seg_cv_img = cv2.cvtColor(np.asarray(colored_preds),
69                                         cv2.COLOR_RGB2BGR)
70         pub_image = CvBridge().cv2_to_imgmsg(seg_cv_img, "bgr8"
71                                             )
72
73         pub_image.header.frame_id = 'camera_link'
74         pub_image.header.stamp = rospy.Time.now()
75         image_pub.publish(pub_image)
76         mask = (seg_cv_img[:, :, 0] == 0) & (seg_cv_img[:, :,
77             1] == 0) & (seg_cv_img[:, :, 2] == 0)
78
79         cv_img[mask] = 0
80         pub_image2 = CvBridge().cv2_to_imgmsg(cv_img, "bgr8")
81         pub_image2.header.frame_id = 'camera_link'
82         pub_image2.header.stamp = rospy.Time.now()
83         image_pub2.publish(pub_image2)

```

```

88         #cv2.imshow("seg_cv_img" , seg_cv_img)
89         #cv2.waitKey()
90         count = count+1
91         print(f"Publishing image number", count)
92
93 if __name__ == '__main__':
94     rospy.init_node("deeplab_ros")
95     load_model()
96     image_sub = rospy.Subscriber("/image_conv", rosImage,
97                                 imageCallback, queue_size=10)
98     rospy.spin()

```

Listing A.2: Python code for semantic segmentation

```

1 import os
2 import torch
3 import torch.nn as nn
4 import numpy as np
5 from torchvision import transforms as T
6 from PIL import Image
7 import cv2
8 import matplotlib.pyplot as plt
9 from datasets import Cityscapes
10 import network
11 import utils
12 from ultralytics import YOLO
13 import pdb
14 global count
15 count = 0
16
17 label_counts = {'car': 0, 'truck': 0, 'person': 0, 'motorcycle'

```



```

: 0, 'bicycle': 0, 'bus': 0}
18 def load_model():
19     global model, device
20
21     device = torch.device('cuda' if torch.cuda.is_available()
22                           else 'cpu')
23     model = network.deeplabv3plus_mobilenet(num_classes=19,
24                                             output_stride=16)
25     utils.set_bn_momentum(model.backbone, momentum=0.01)
26     ckpt = "/home/tarun/Documents/deeplab/model/
27           best_deeplabv3plus_mobilenet_cityscapes_os16.pth"
28     if os.path.isfile(ckpt):
29         checkpoint = torch.load(ckpt, map_location=torch.device
30                                ('cpu'))
31         model.load_state_dict(checkpoint["model_state"])
32         model = nn.DataParallel(model)
33         model.to(device)
34         print("Resume model from %s" % ckpt)
35         del checkpoint
36     else:
37         print("[!] Retrain")
38         model = nn.DataParallel(model)
39         model.to(device)
40     model = model.eval()
41
42 class CustomDataset:
43     def __init__(self, data_dir, transform=None):
44         self.data_dir = data_dir
45         self.transform = transform
46         self.images = os.listdir(data_dir)

```

```

43
44     def __len__(self):
45         return len(self.images)
46
47     def __getitem__(self, idx):
48         image_path = os.path.join(self.data_dir, self.images[
49             idx])
50         image = Image.open(image_path)
51         if self.transform:
52             image = self.transform(image)
53         return image
54
55     def process_image(image):
56         global device, count, label_counts, model_yolo
57         counts = {'car': 0, 'truck': 0, 'person': 0, 'motorcycle':
58             0, 'bicycle': 0, 'bus': 0}
59
60         results = model_yolo(image)
61         class_list = results[0].boxes.cls.cpu().numpy()
62         counts['car'] = np.sum(results[0].boxes.cls.cpu().numpy()
63             == 2)
64         counts['truck'] = np.sum(results[0].boxes.cls.cpu().numpy()
65             == 8)
66         counts['person'] = np.sum(results[0].boxes.cls.cpu().numpy()
67             == 0)
68         counts['motorcycle'] = np.sum(results[0].boxes.cls.cpu().
69             numpy() == 3)
70         counts['bicycle'] = np.sum(results[0].boxes.cls.cpu().numpy()
71             == 1)
72         counts['bus'] = np.sum(results[0].boxes.cls.cpu().numpy()
73             == 5)

```

```

65     transform = T.Compose([
66         T.ToTensor(),
67         T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
68             0.224, 0.225])),
69     ])
70     with torch.no_grad():
71         image = transform(image).unsqueeze(0)
72         image = image.to(device)
73         pred = model(image)
74         pred = pred.squeeze(0).cpu().numpy()
75         pred_class = np.argmax(pred, axis=0)
76         for label in counts.keys():
77             label_counts[label] += counts[label]
78
79         count = count + 1
80         print(f"Processing image number {count}")
81 if __name__ == '__main__':
82     load_model()
83     data_dir = "/home/tarun/Documents/diss_deepvo/DeepVO-
84         pytorch/DeepVO-pytorch-master/KITTI/images/04"
85     custom_dataset = CustomDataset(data_dir)
86
87     model_yolo = YOLO('yolov8n.pt')
88     for image in custom_dataset:
89         process_image(image)
90
91
92

```

```
93     for label_name, count in label_counts.items():
94         print(f"Total count of {label_name}: {count}")
95
96     labels = list(label_counts.keys())
97     counts = list(label_counts.values())
98     plt.bar(labels, counts)
99     plt.title('Total Label Counts')
100    plt.xlabel('Labels')
101    plt.ylabel('Count')
102    plt.show()
```

Listing A.3: Python code for getting stastical data

References

- [1] R. Mur-Artal and J. D. Tardós, Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras, *IEEE Transactions on Robotics* [online], vol. 33, no. 5 2017, pp. 1255–1262, 2017. DOI: 10.1109/TR0.2017.2705103.
- [2] C. Wang, G. Wang, P. Li, and X. Sun, Deepvo: Towards end-to-end visual odometry with deep recurrent convolutional neural networks, *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 12 2017, pp. 3738–3752, 2017.
- [3] L.-C. Chen *et al.*, “Encoder-decoder with atrous separable convolution for semantic image segmentation,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 801–818.
- [4] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for autonomous driving? the kitti vision benchmark suite,” in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012.
- [5] J. Leonard and H. Durrant-Whyte, Mobile robot localization by tracking geometric beacons, *IEEE Transactions on Robotics and Automation*, vol. 7, no. 3 Jun. 1991, pp. 376–382, Jun. 1991.
- [6] *Robotics and Autonomous Systems*, vol. 6, no. 1-2 Jun. 1990, p. 193, Jun. 1990.
- [7] A. J. Davison, “Real-time simultaneous localisation and mapping with a single camera,” in *Proceedings of IEEE Conference*, 2003, p. 140.
- [8] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, Orb-slam: A versatile and accurate monocular slam system, *IEEE Transactions on Robotics* [online], vol. 31, no. 5 2015, pp. 1147–1163, 2015. DOI: 10.1109/TR0.2015.2463671.
- [9] J. Fuentes-Pacheco, J. Ruiz-Ascencio, and J. M. Rendón-Mancha, *Visual simultaneous localization and mapping: A survey*, Nov. 2012.

- [10] S. Leutenegger, S. Lynen, M. Bosse, R. Siegwart, and P. Furgale, Keyframe-based visual–inertial odometry using nonlinear optimization, *The International Journal of Robotics Research*, vol. 34, no. 3 2015, pp. 314–334, 2015.
- [11] B. Triggs, P. F. McLauchlan, R. Hartley, and A. Fitzgibbon, *Bundle Adjustment — A Modern Synthesis*, https://doi.org/10.1007/3-540-44480-7_21, Jan. 2000.
- [12] Y. Chen, Y. Zhou, Q. Lv, and K. K. Deveerasetty, “A review of v-slam,” in *2018 IEEE International Conference on Information and Automation (ICIA)*, 2018, pp. 603–608.
- [13] A. Macario Barros, M. Michel, Y. Moline, G. Corre, and F. Carrel, A comprehensive survey of visual slam algorithms, *Robotics*, vol. 11, no. 1 2022, 2022, ISSN: 2218-6581.
- [14] S. Li, D. Zhang, Y. Xian, B. Li, T. Zhang, and C. Zhong, Overview of deep learning application on visual slam, vol. 74 2022, p. 102 298, 2022, ISSN: 0141-9382.
- [15] C. Cadena, L. Carlone, H. Carrillo, *et al.*, Past, present, and future of simultaneous localization and mapping: Towards the robust-perception age, *IEEE Transactions on Robotics*, vol. 32, no. 6 2016, pp. 1309–1332, 2016.
- [16] D. Nister, O. Naroditsky, and J. Bergen, Visual odometry, *eng*, vol. 1 2004, pp. I–I, 2004.
- [17] F. Fraundorfer and D. Scaramuzza, Visual odometry: Part ii - matching, robustness, optimization, and applications, *IEEE Robot. Autom. Mag.*, vol. 19, no. 2 2012, pp. 78–90, 2012.
- [18] A. MO, M. MH, S. MI, and I. NB, Review of visual odometry: Types, approaches, challenges, and applications, *Springerplus*, vol. 5, no. 1 Oct. 2016, p. 1897, Oct. 2016.
- [19] D. Scaramuzza and F. Fraundorfer, Tutorial: Visual odometry, *IEEE Robot Autom Mag*, vol. 18, no. 4 2011, pp. 80–92, 2011.
- [20] D. V. García, L. F. Rojo, A. Gil, L. P. Castelló, and S. R. García, *Visual Odometry through Appearance- and Feature-Based Method with Omnidirectional Images*, Jan. 2012.
- [21] D. Scaramuzza and F. Fraundorfer, Visual odometry [tutorial], *IEEE Robotics & Automation Magazine*, vol. 18, no. 4 2011, pp. 80–92, 2011, [Pages 1, 8, 12, 13, and 16.]
- [22] D. G. Lowe, “Object recognition from local scale-invariant features,” in *Proceedings of the Seventh IEEE International Conference on Computer Vision*, [Page 13.], vol. 2, 1999, pp. 1150–1157.

- [23] D. G. Lowe, Distinctive image features from scale-invariant keypoints, *International Journal of Computer Vision*, vol. 60, no. 2 2004, pp. 91–110, 2004, [Page 13.]
- [24] E. Rosten and T. Drummond, “Machine learning for high-speed corner detection,” in *European Conference on Computer Vision*, [Page 13.], Springer, 2006, pp. 430–443.
- [25] R. Szeliski, *Computer Vision: Algorithms and Applications* (Texts in Computer Science Ser.), 2nd. Springer, Cham, 2022, [Pages 14 and 17.], ISBN: 978-3-030-34372-9.
- [26] Q. Liu, Z. Wang, and H. Wang, Optical flow monocular visual-inertial odometry with online photometric calibration, *Journal of Physics: Conference Series*, vol. 1828, no. 1 Feb. 2021, p. 012 163, Feb. 2021.
- [27] A. Handa, M. Bloesch, V. Patraucean, S. Stent, J. McCormac, and A. Davison, “Gvnn: Neural network library for geometric computer vision,” in *arXiv:1607.07405*, 2016.
- [28] Y. Guo, Y. Liu, A. Oerlemans, S. Lao, S. Wu, and M. S. Lew, Deep learning for visual understanding: A review, *Neurocomputing*, vol. 187 2015, pp. 27–48, 2015.
- [29] Y. LeCun, Y. Bengio, and G. Hinton, Deep learning, *Nature* 2016, 2016.
- [30] Q. Liu, H. Zhang, Y. Xu, and L. Wang, Unsupervised deep learning-based rgb-d visual odometry, *Applied Sciences*, vol. 10, no. 16 2020, 2020, ISSN: 2076-3417.
- [31] S. Wang, R. Clark, H. Wen, and N. Trigoni, End-to-end, sequence-to-sequence probabilistic visual odometry through deep neural networks, *The International Journal of Robotics Research*, vol. 37, no. 4-5 2018, pp. 513–542, 2018.
- [32] A. Kendall and R. Cipolla, “Modelling uncertainty in deep learning for camera relocalization,” in *Proceedings of the 2016 IEEE International Conference on Robotics and Automation (ICRA)*, Stockholm, Sweden, May 2016, pp. 4762–4769.
- [33] Wikipedia. “Scale-invariant feature transform.” (), available from: https://en.wikipedia.org/wiki/Scale-invariant_feature_transform [Accessed 10/17/2023].
- [34] “Opencv: Introduction to SIFT (Scale-Invariant Feature Transform).” (), available from: https://docs.opencv.org/4.x/da/df5/tutorial_py_sift_intro.html [Accessed 10/17/2023].

- [35] J. Hartmann, J. H. Klüssendorff, and E. Maehle, “A comparison of feature descriptors for visual slam,” in *2013 European Conference on Mobile Robots (ECMR)*, Barcelona, Spain, Sep. 2013.
- [36] H. Bay, A. Ess, T. Tuytelaars, and L. V. Gool, Surf: Speeded up robust features, *Computer Vision and Image Understanding (CVIU)*, vol. 110, no. 3 2008, pp. 346–359, 2008.
- [37] *Opencv: Introduction to surf (speeded-up robust features)*. available from: https://docs.opencv.org/3.4/df/dd2/tutorial_py_surf_intro.html [Accessed 10/17/2023].
- [38] Wikipedia, *Speeded up robust features*. available from: https://en.wikipedia.org/wiki/Speeded_up_robust_features [Accessed 10/17/2023].
- [39] M. Calonder, V. Lepetit, C. Strecha, and P. Fua, “Brief: Binary robust independent elementary features,” in *Computer Vision – ECCV 2010*, K. Daniilidis, P. Maragos, and N. Paragios, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 778–792, ISBN: 978-3-642-15561-1.
- [40] E. Rosten and T. Drummond, “Machine learning for high-speed corner detection,” in *European Conf. on Computer Vision (ECCV)*, vol. 1, May 2006, pp. 430–443.
- [41] M. Muja and D. G. Lowe, “Fast approximate nearest neighbors with automatic algorithm configuration,” in *Int. Conf. on Computer Vision Theory and Application*, 2009, pp. 331–340.
- [42] A. Gionis, P. Indyk, and R. Motwani, “Similarity search in high dimensions via hashing,” in *Proc. of the Int. Conf. on Very Large Data Bases (VLDB)*, 1999, pp. 518–529.
- [43] M. Kumar and S. Dargan, A survey of deep learning and its applications: A new paradigm to machine learning, *Archives of Computational Methods in Engineering* Jun. 2019, Jun. 2019.
- [44] R. E. Uhrig, “Introduction to artificial neural networks,” in *Proceedings of IECON '95 - 21st Annual Conference on IEEE Industrial Electronics*, vol. 1, Nov. 1995, 33–37 vol.1.
- [45] K. Fukushima, Neocognitron: A hierarchical neural network capable of visual pattern recognition, *Neural Networks*, vol. 1, no. 2 1988, pp. 119–130, 1988.
- [46] Y. LeCun *et al.*, Backpropagation applied to handwritten zip code recognition, *Neural Computation*, vol. 1, no. 4 1989, pp. 541–551, 1989.

- [47] A. Garcia-Garcia *et al.*, A review on deep learning techniques applied to semantic segmentation, *arXiv preprint arXiv:1704.06857* 2017, 2017.
- [48] R. Yamashita *et al.*, Convolutional neural networks: An overview and application in radiology, *Insights into Imaging*, vol. 9, no. 4 2018, pp. 611–629, 2018.
- [49] R. Yamashita *et al.*, Convolutional neural networks: An overview and application in radiology, *Insights into Imaging*, vol. 9, no. 4 2018, pp. 611–629, 2018.
- [50] K. Hu, F. Zheng, L. Weng, Y. Ding, and J. Jin, Action recognition algorithm of spatio-temporal differential lstm based on feature enhancement, *Appl. Sci.*, vol. 11 2021, p. 7876, 2021.
- [51] A. Graves, Supervised sequence labelling, in *Supervised Sequence Labelling with Recurrent Neural Networks*. Springer, 2012, pp. 5–13, [Pages 20 and 21.]
- [52] R. Kuemmerle, G. Grisetti, H. Strasdat, K. Konolige, and W. Burgard, “G2o: A general framework for graph optimization,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, Shanghai, China, May 2011, pp. 3607–3613. DOI: 10.1109/ICRA.2011.5979949.
- [53] D. Filliat, “A visual bag of words method for interactive qualitative localization and mapping,” in *Proceedings 2007 IEEE International Conference on Robotics and Automation*, Apr. 2007, pp. 3921–3926. DOI: 10.1109/ROBOT.2007.364080.
- [54] H. Strasdat, A. J. Davison, J. M. M. Montiel, and K. Konolige, “Double window optimization for constant time visual slam,” in *IEEE International Conference on Computer Vision (ICCV)*, Barcelona, Spain, Nov. 2011, pp. 2352–2359.
- [55] C. Mei, G. Sibley, and P. Newman, “Closing loops without places,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Taipei, Taiwan, Oct. 2010, pp. 3738–3744.
- [56] R. Mur-Artal and J. D. Tardos, “Fast relocalisation and loop closing in keyframe-based slam,” in *IEEE International Conference on Robotics and Automation (ICRA)*, Hong Kong, China, Jun. 2014, pp. 846–853.

- [57] V. Lepetit, F. Moreno-Noguer, and P. Fua, Epnnp: An accurate $O(n)$ solution to the pnp problem, *International Journal of Computer Vision*, vol. 81, no. 2 2009, pp. 155–166, 2009.
- [58] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2003.
- [59] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd. Cambridge University Press, 2004.
- [60] O. D. Faugeras and F. Lustman, Motion and structure from motion in a piecewise planar environment, *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 2, no. 03 1988, pp. 485–508, 1988.
- [61] J. Donahue, L. A. Hendricks, S. Guadarrama, *et al.*, Long-term recurrent convolutional networks for visual recognition and description, *IEEE Transactions on Pattern Analysis and Machine Intelligence* to appear, to appear.
- [62] A. Dosovitskiy, P. Fischer, E. Ilg, *et al.*, “FlowNet: Learning optical flow with convolutional networks,” in *Proceedings of IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [63] B. Cheng, A. G. Schwing, and A. Kirillov, *Per-pixel classification is not all you need for semantic segmentation*, 2021. arXiv: 2107.06278 [cs.CV].
- [64] W. S. McCulloch and W. Pitts, A logical calculus of the ideas immanent in nervous activity, *The Bulletin of Mathematical Biophysics* [online], vol. 5, no. 4 1943, pp. 115–133, 1943, ISSN: 1522-9602. DOI: 10.1007/BF02478259. available from: <https://doi.org/10.1007/BF02478259>.
- [65] L.-C. Chen *et al.*, Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 4 2017, pp. 834–848, 2017.
- [66] L.-C. Chen *et al.*, “Encoder-decoder with atrous separable convolution for semantic image segmentation,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 801–818.

- [67] L.-C. Chen *et al.*, Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 4 2018, pp. 834–848, 2018.
- [68] L.-C. Chen *et al.*, Rethinking atrous convolution for semantic image segmentation, *arXiv preprint arXiv:1706.05587* 2017, 2017.
- [69] raulmur, *Github - raulmur/ORB_SLAM2: Real-Time SLAM for Monocular, Stereo and RGB-D Cameras, with Loop Detection and Relocalization Capabilities*. available from: https://github.com/raulmur/ORB_SLAM2/tree/master [Accessed 10/19/2023].
- [70] , *Github - ChiWeiHsiao/DeepVO-pytorch: Pytorch Implementation of DeepVO*. available from: <https://github.com/ChiWeiHsiao/DeepVO-pytorch> [Accessed 10/19/2023].
- [71] *Pre trained model for deepvo on kitti dataset*. available from: <https://drive.google.com/file/d/1l0s3rYWgN8bL0Fyofee8IhN-0knxJF22/view?pli=1> [Accessed 10/19/2023].
- [72] *Pre trained deeplab v3+ on cityspace dataset*. available from: https://www.dropbox.com/s/753ojyvsh3vdjol/best_deeplabv3plus_mobilenet_cityscapes_os16.pth?dl=0 [Accessed 10/19/2023].
- [73] , *Github - VainF/DeepLabV3Plus-Pytorch: Pretrained DeepLabv3 and DeepLabv3+ for Pascal VOC Cityscapes*. available from: <https://github.com/VainF/DeepLabV3Plus-Pytorch#segmentation-results-on-cityscapes-deeplabv3plus-mobilenet> [Accessed 10/19/2023].
- [74] P. Wang, R. Yang, B. Cao, W. Xu, and Y. Lin, “Dels-3d: Deep localization and segmentation with a 3d semantic map,” in *CVPR*, 2018, pp. 5860–5869.
- [75] J. Jeong, Y. Cho, Y.-S. Shin, H. Roh, and A. Kim, Complex urban dataset with multi-level sensors from highly diverse urban environments, *International Journal of Robotics Research*, vol. 38, no. 6 2019, pp. 642–657, 2019.